



Deep Learning Framework

Zetgoo Team

zetgoo.cs@gmail.com

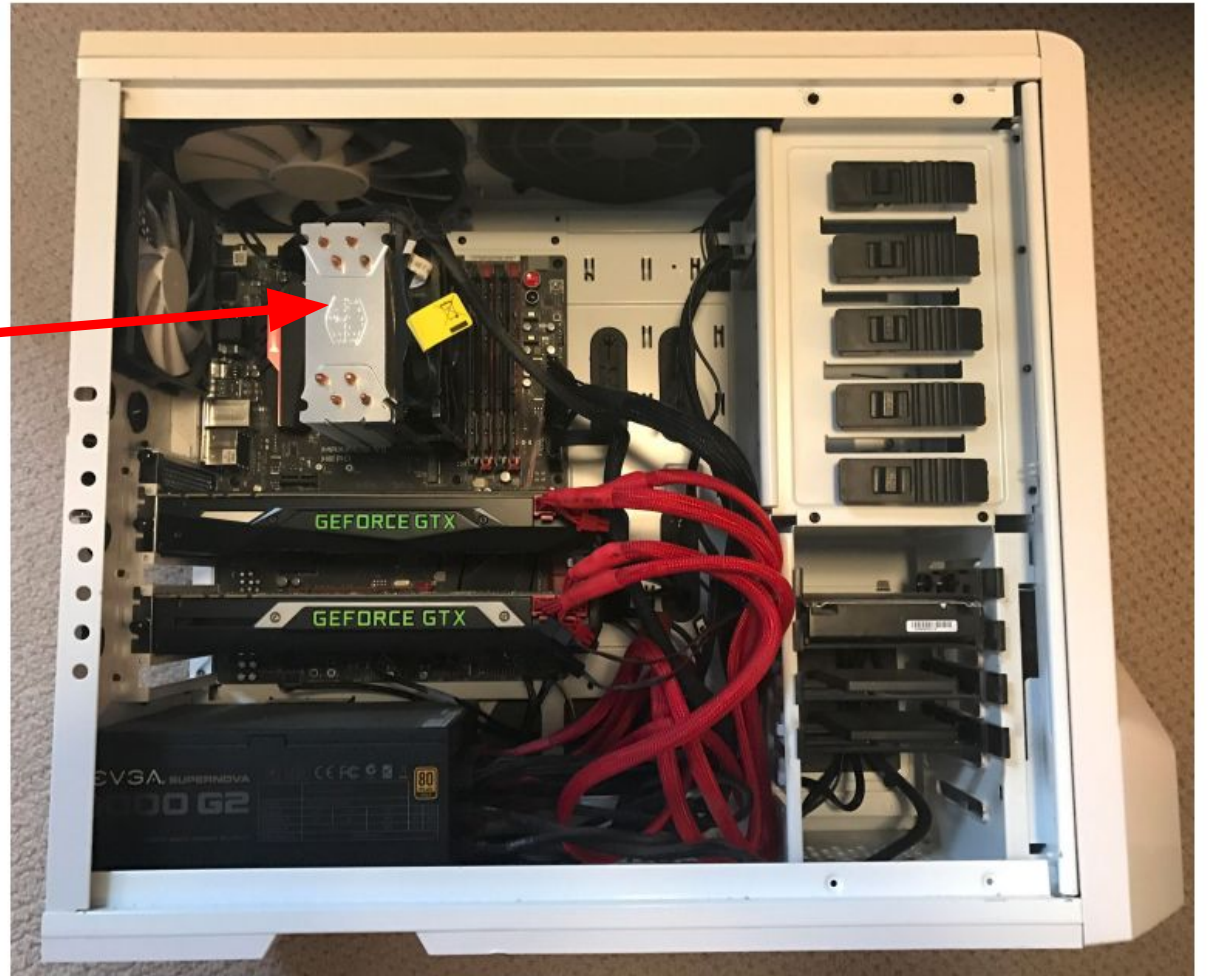
Inside a computer



Inside a computer



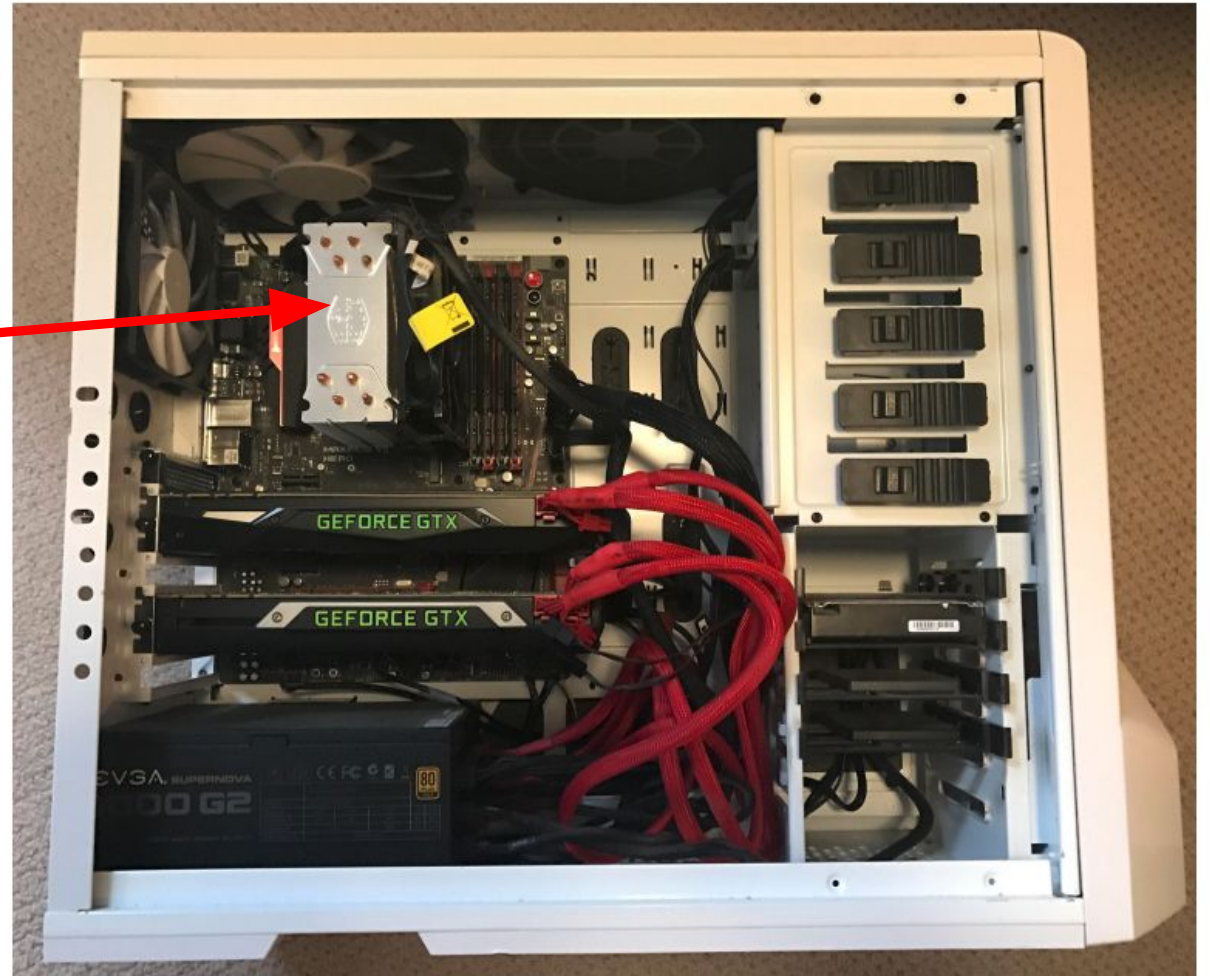
Spot the CPU!



Inside a computer



Spot the GPU!



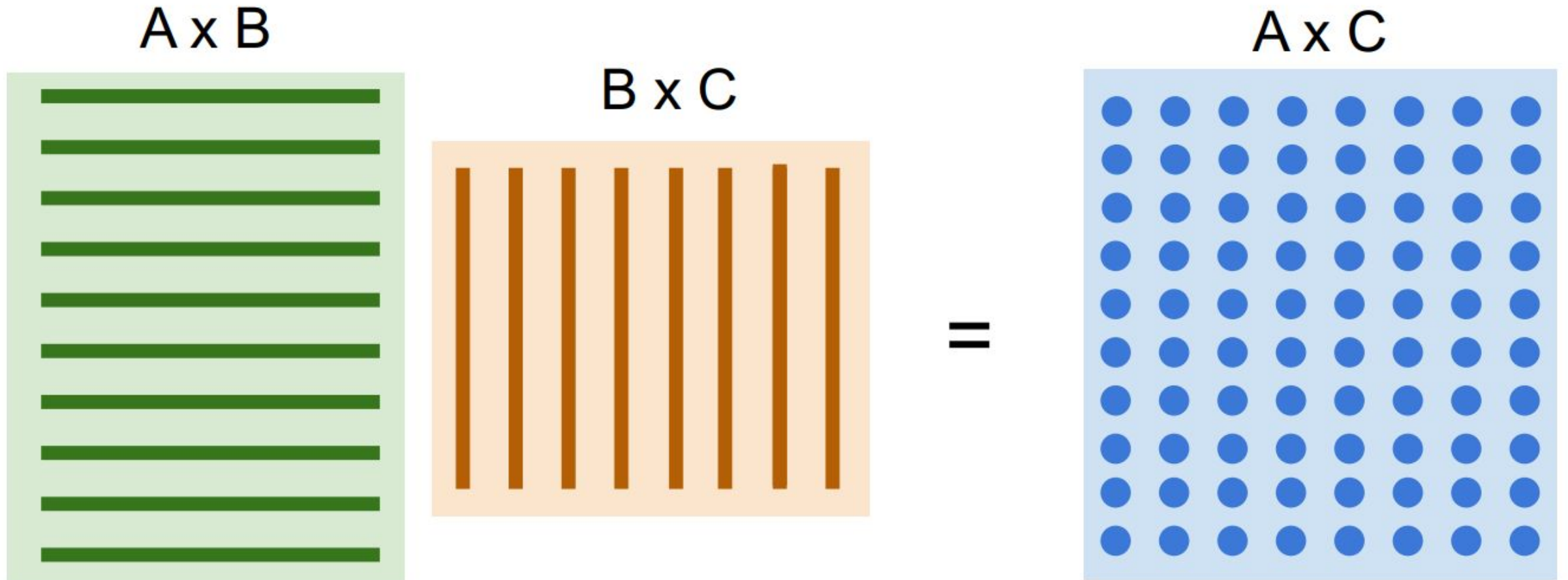
CPU vs GPU

	Cores	Clock Speed	Memory	Price	Speed
CPU (Intel Core i7-7700k)	4 (8 threads with hyperthreading)	4.2 GHz	System RAM	\$385	~540 GFLOPs FP32
GPU (NVIDIA RTX 2080 Ti)	3584	1.6 GHz	11 GB GDDR6	\$1199	~13.4 TFLOPs FP32

CPU: Fewer cores, but each core is much faster and much more capable; great at sequential tasks

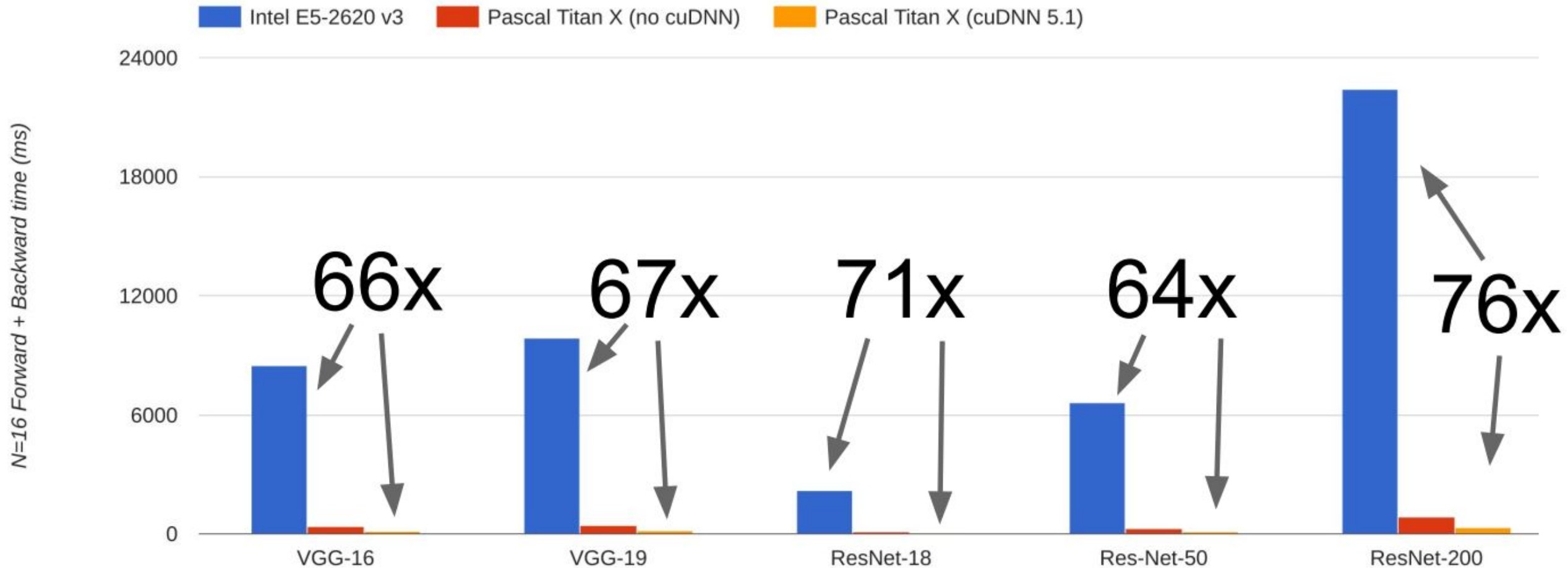
GPU: More cores, but each core is much slower and “dumber”; great for parallel task

Example: Matrix Multiplication



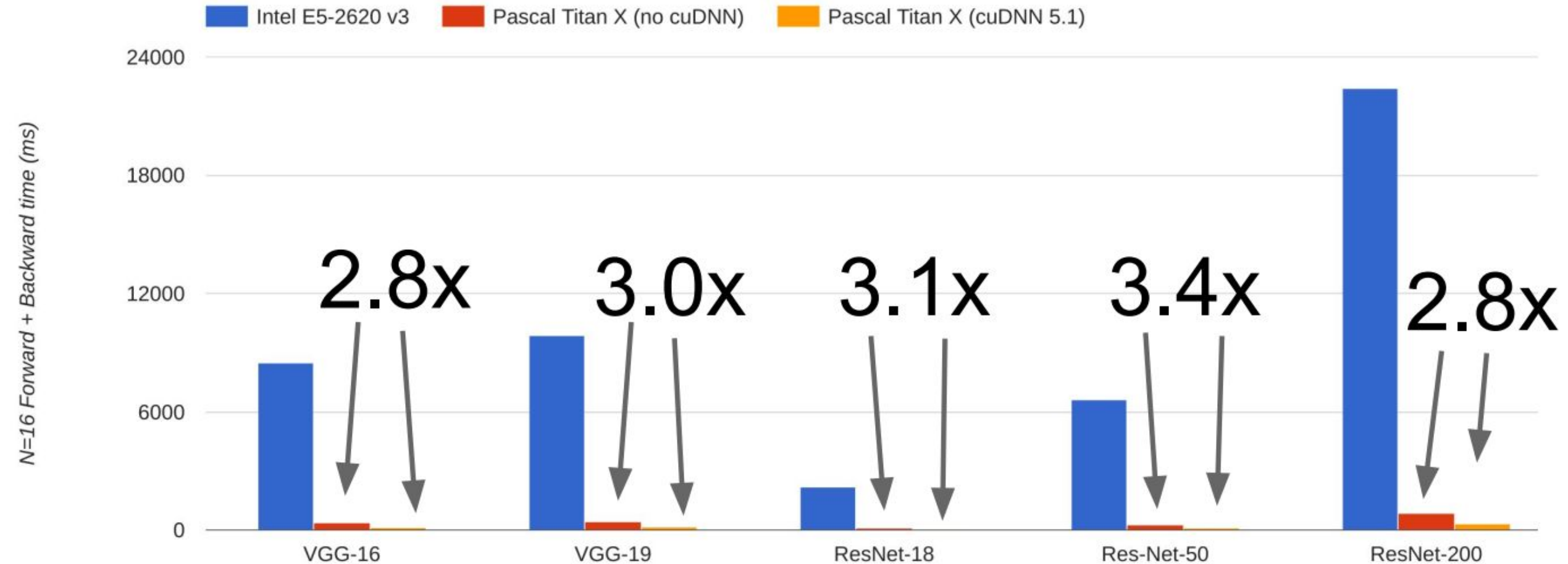
CPU vs GPU in Practice

(CPU performance not well-optimized, a little unfair)



CPU vs GPU in Practice

cuDNN much faster than
“unoptimized” CUDA



CPU vs GPU

	Cores	Clock Speed	Memory	Price	Speed
CPU (Intel Core i7-7700k)	4 (8 threads with hyperthreading)	4.2 GHz	System RAM	\$385	~540 GFLOPs FP32
GPU NVIDIA RTX 2080 Ti	3584	1.6 GHz	11 GB GDDR6	\$1099	~13 TFLOPs FP32 ~114 TFLOPs FP16
GPU (Data Center) NVIDIA V100	5120 CUDA, 640 Tensor	1.5 GHz	16/32 GB HBM2	\$2.5/hr (GCP)	~8 TFLOPs FP64 ~16 TFLOPs FP32 ~125 TFLOPs FP16
TPU Google Cloud TPUv3	2 Matrix Units (MXUs) per core, 4 cores	?	128 GB HBM	\$8/hr (GCP)	~420 TFLOPs (non-standard FP)

CPU: Fewer cores, but each core is much faster and much more capable; great at sequential tasks

GPU: More cores, but each core is much slower and “dumber”; great for parallel task

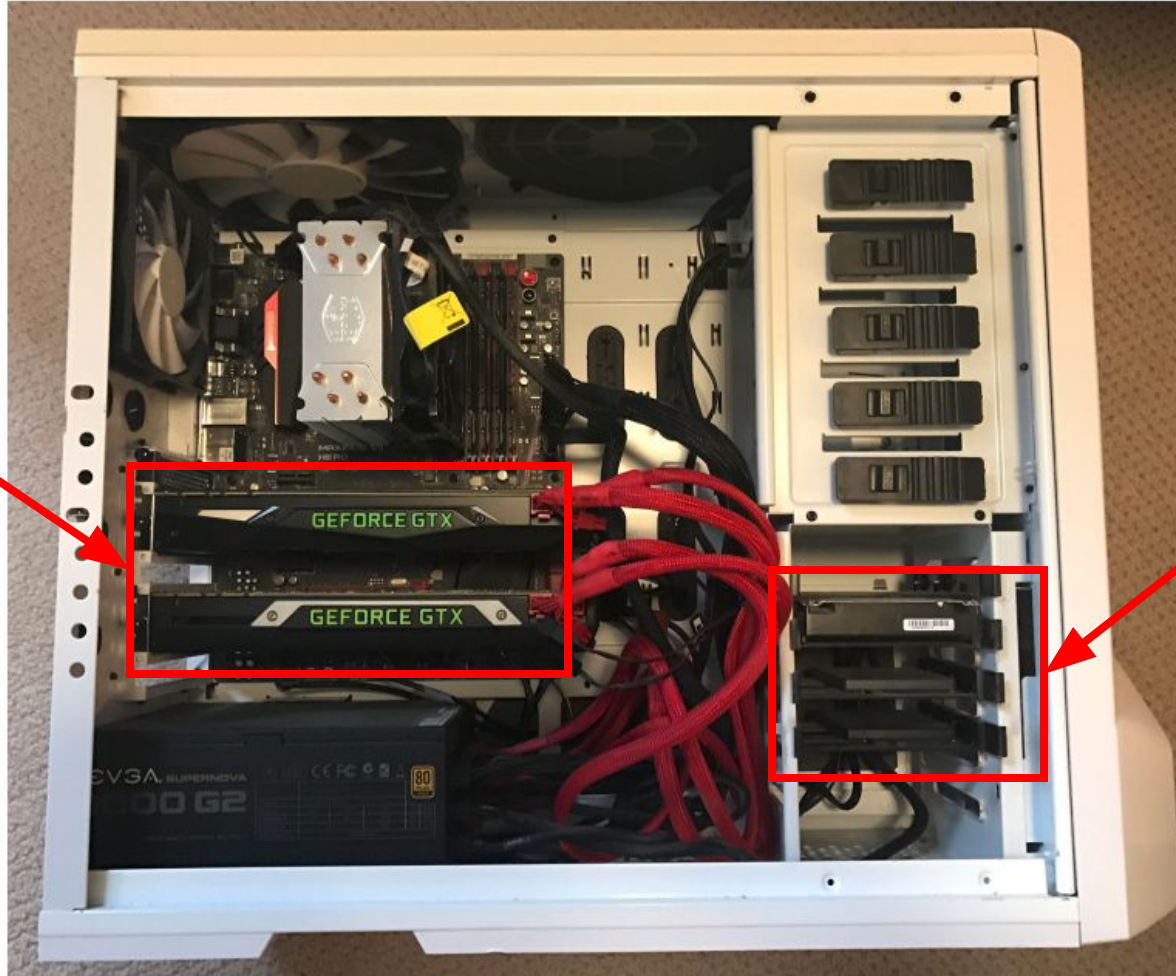
TPU: Specialized hardware for deep learning

Programming GPUss

- CUDA (NVIDIA only)
 - Write C-like code that runs directly on the GPU
 - Optimized APIs: cuBLAS, cuFFT, cuDNN, etc
- OpenCL
 - Similar to CUDA, but runs on anything
 - Usually slower on NVIDIA hardware
- HIP <https://github.com/ROCm-Developer-Tools/HIP>
 - New project that automatically converts CUDA code to something that can run on AMD GPUs

CPU vs GPU Communication

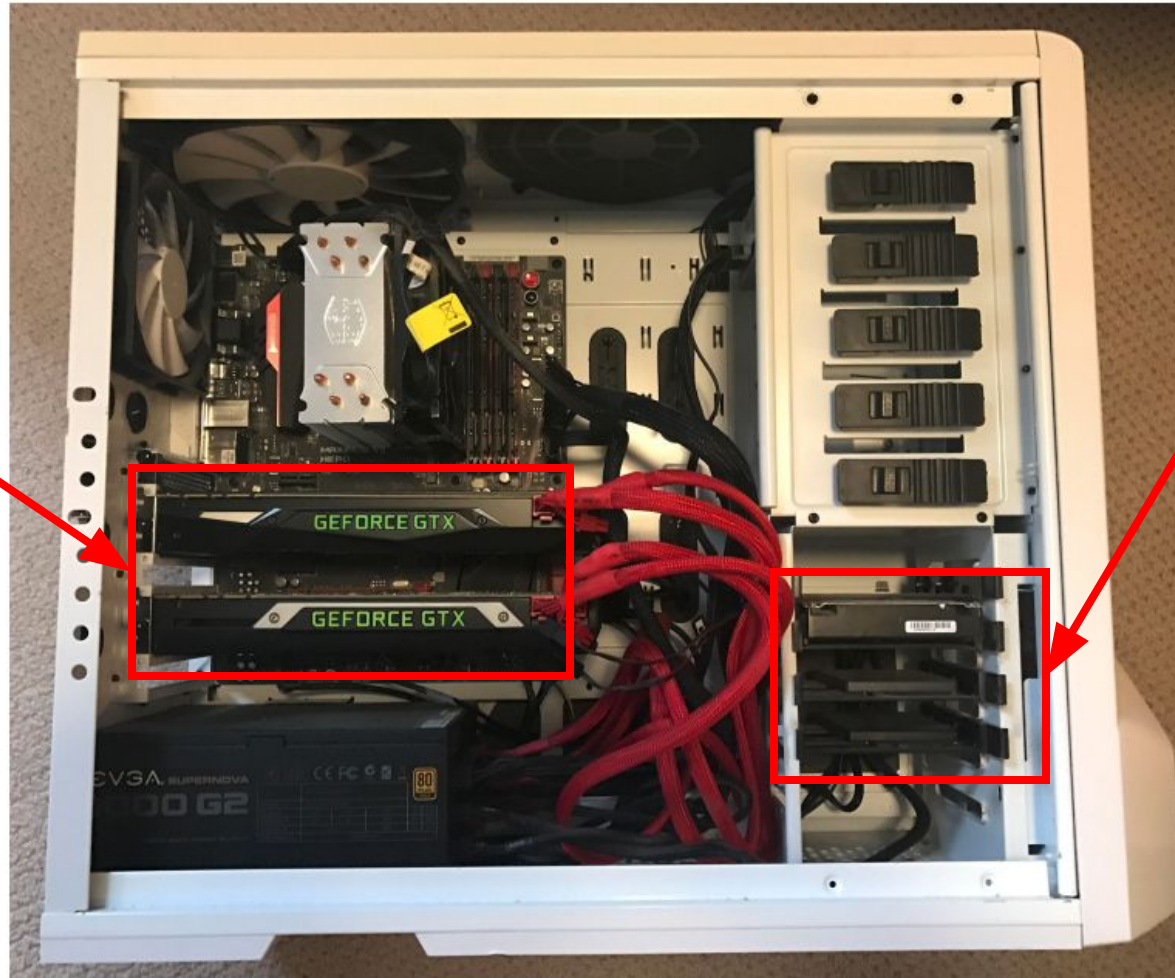
Model is
here



Data is
here

CPU vs GPU Communication

Model is
here

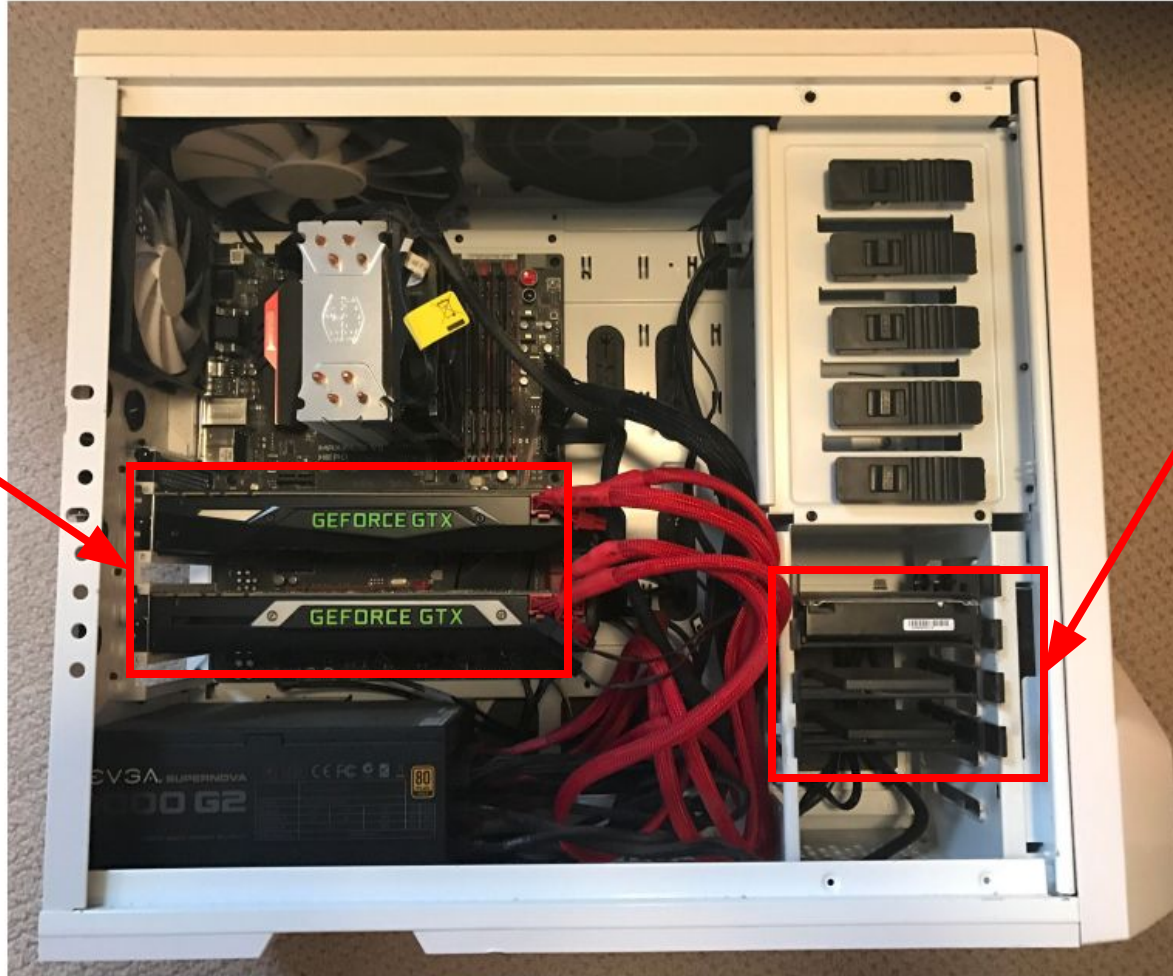


Data is
here

If you aren't careful, training
can bottleneck on reading
data and transferring to GPU!

CPU vs GPU Communication

Model is
here



Data is
here

If you aren't careful, training can bottleneck on reading data and transferring to GPU!

Solutions:

- Read all data into RAM
- Use SSD instead of HDD
- Use multiple CPU threads to prefetch data



Deep Learning Software

Deep learning and Its Applications in Python

Before 2017

Caffe
(UC Berkeley)

Torch
(NYU / Facebook)

Theano
(U Montreal)



TensorFlow
(Google)

From 2017

Caffe
(UC Berkeley)



Caffe 2
(Facebook)

Torch
(NYU / Facebook)



PyTorch
(Facebook)

Theano
(U Montreal)



TensorFlow
(Google)

Chainer

(Preferred Networks)

The company has officially migrated its research infrastructure to PyTorch

Paddle
(Baidu)

CNTK
(Microsoft)

MXNet
(Amazon)

Developed by U Washington, CMU, MIT, Hong Kong U, etc but main framework of choice at AWS

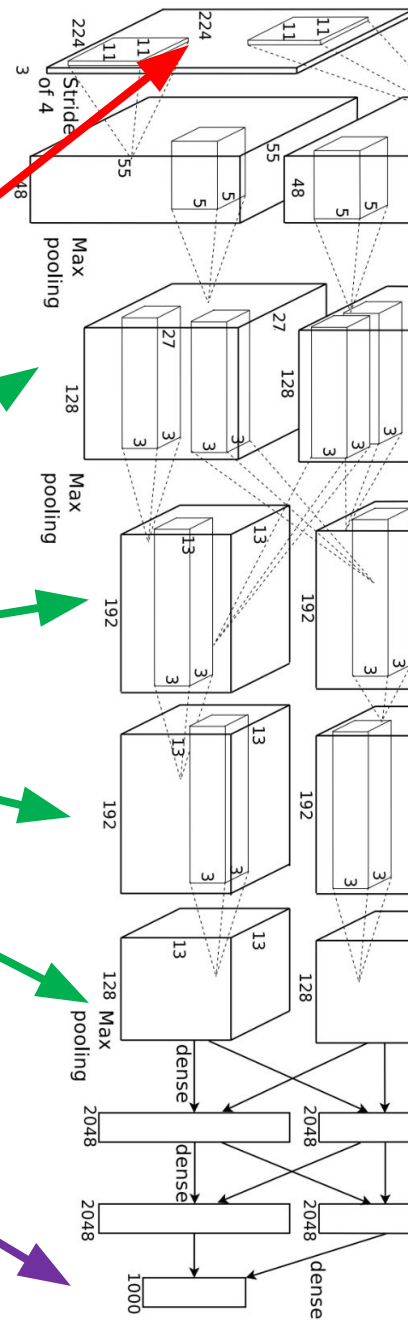
And
others

Computational Graphs

Input Image

Weights

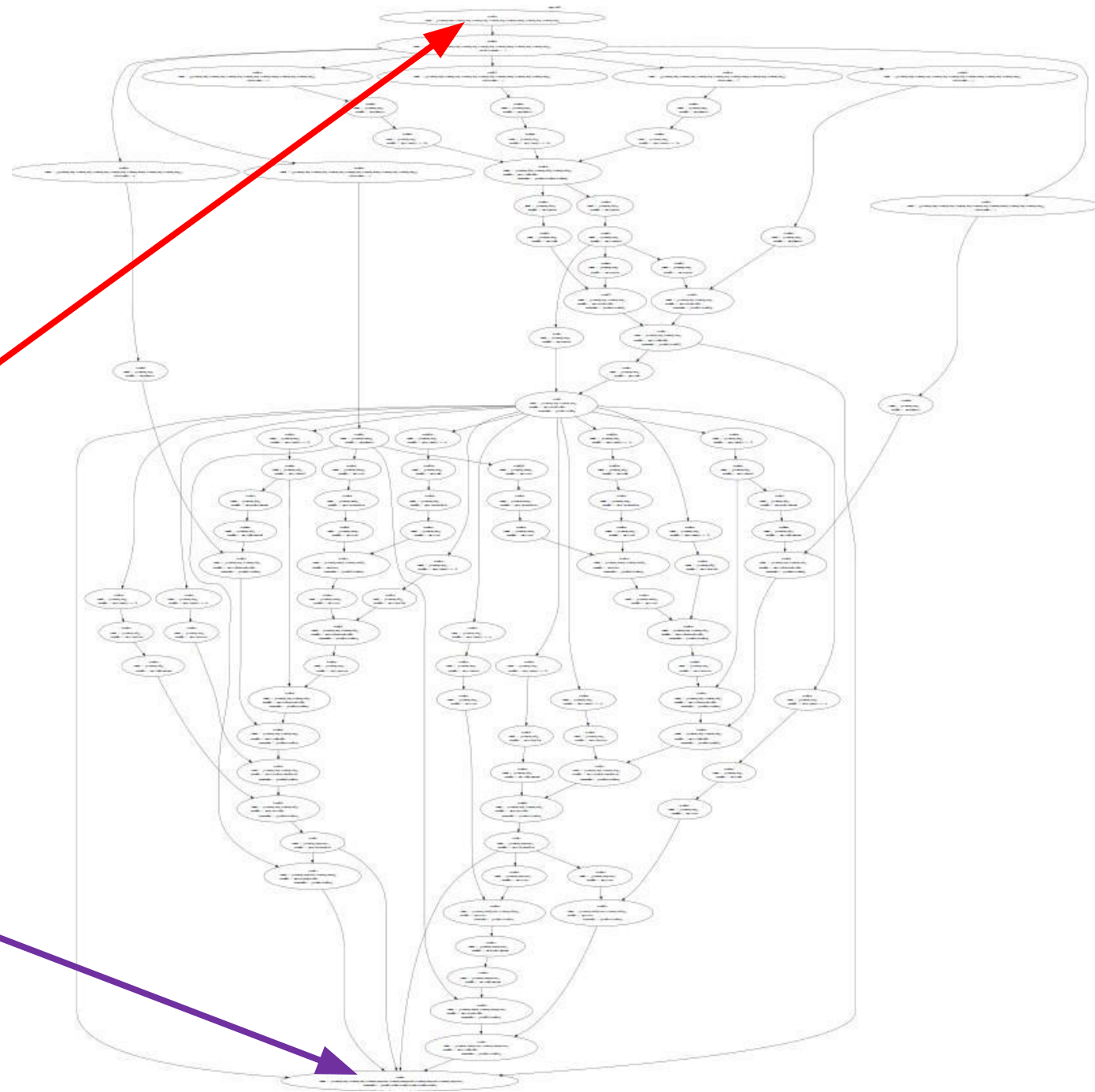
loss

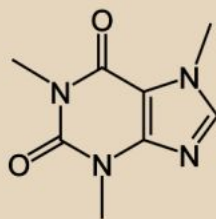


Computational Graphs

Input Image

loss





Caffe:

Convolutional Architecture for Fast Feature Embedding

Created by Yangqing Jia

Developed by BVLC

caffe.berkeleyvision.org

bvlc.eecs.berkeley.edu



Caffe

Deep learning and Its Applications in Python



What is Caffe?

- Pure C++ / CUDA architecture for deep learning
 - Command line, Python, MATLAB interfaces
- Fast, well-tested code
- Tools, reference models, demos, and recipes
- Seamless switch between CPU and GPU



Prototype



Training



Deployment

All with essentially the same code!



Parameters in Caffe

- `caffe.proto`
 - Generate classes to pass the parameters
- Parameters are listed in order



Reference Models

- Caffe offers the
 - model definitions
 - optimization settings
 - pre-trained weights
- so you can start right away.
- The BVLC models are licensed for unrestricted use.



Open Model Collection

- The Caffe [Model Zoo](#)
- open collection of deep models to share innovation
- VGG ILSVRC14 + Devil models **in the zoo**
- Network-in-Network / CCCP model **in the zoo**
- MIT Places scene recognition model **in the zoo**
- help disseminate and reproduce research
- bundled tools for loading and publishing models
- **Share Your Models!** with your citation + license of course

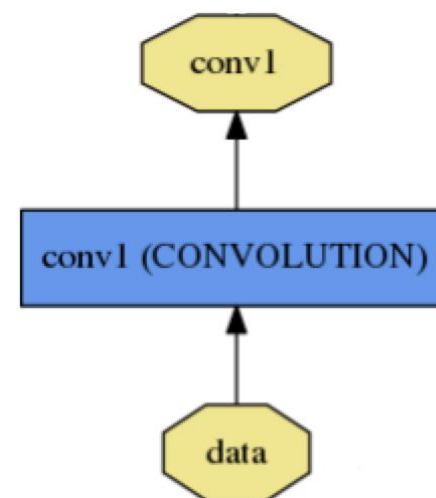


A Caffe Layer

```
name: "conv1"
type: CONVOLUTION
bottom: "data"
top: "conv1"
convolution_param {
  num_output: 20
  kernel_size: 5
  stride: 1
  weight_filler {
    type: "xavier"
  }
}
```

name, type, and the connection structure (input blobs and output blobs)

layer-specific parameters



- Every layer type defines
 - Setup
 - Forward
 - Backward

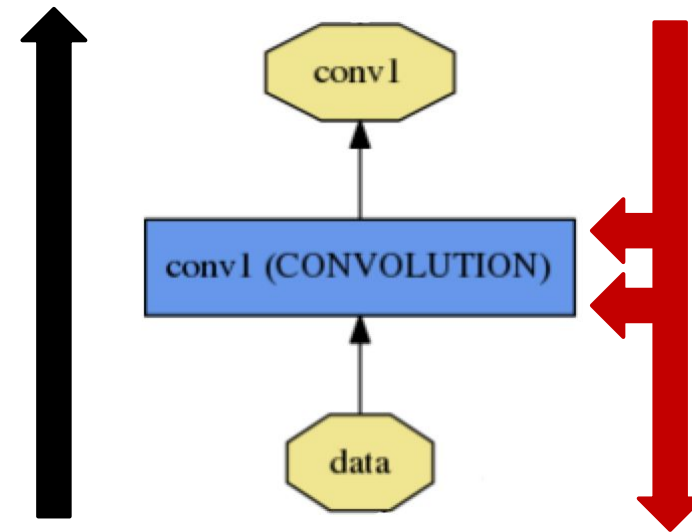


Layer Protocol

Setup: run once for initialization.

Forward: make output given input.

Backward: make gradient of output
- w.r.t. bottom
- w.r.t. parameters (if needed)



Model Composition

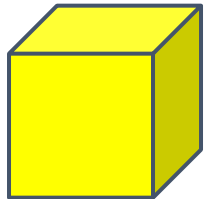
The Net forward and backward passes are the composition the layers'.

Blob



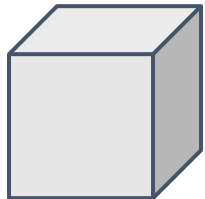
Blobs are 4-D arrays for storing and communicating information.

- hold data, derivatives, and parameters
- lazily allocate memory
- shuttle between CPU and GPU



Data

Number x K Channel x Height x Width
256 x 3 x 227 x 227 for ImageNet train input



Parameter: Convolution Weight

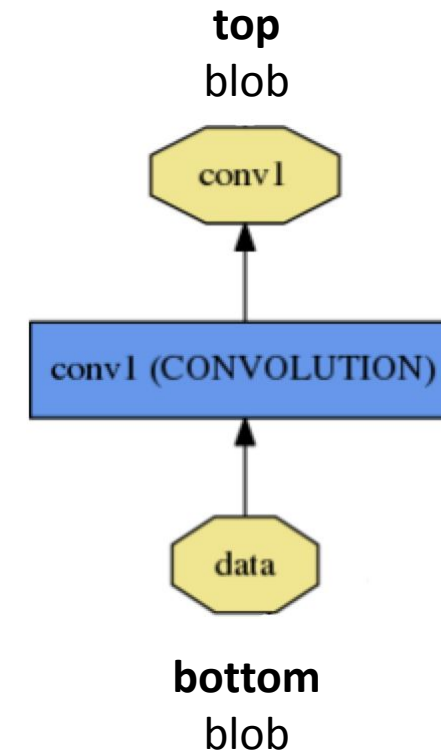
N Output x K Input x Height x Width
96 x 3 x 11 x 11 for CaffeNet conv1



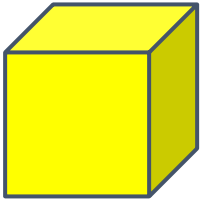
Parameter: Convolution Bias

96 x 1 x 1 x 1 for CaffeNet conv1

```
name: "conv1"  
type: CONVOLUTION  
bottom: "data"  
top: "conv1"  
... definition ...
```



Blob



Reshape(num, channel, height, width)

- declare dimensions
- make *SyncedMem* -- but only lazily allocate

cpu_data(), mutable_cpu_data()

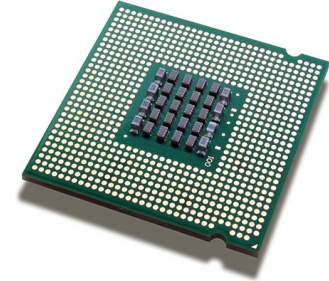
- host memory for CPU mode

gpu_data(), mutable_gpu_data()

- device memory for GPU mode

{cpu,gpu}_diff(), mutable_{cpu,gpu}_diff()

- derivative counterparts to data methods
- easy access to data + diff in forward / backward

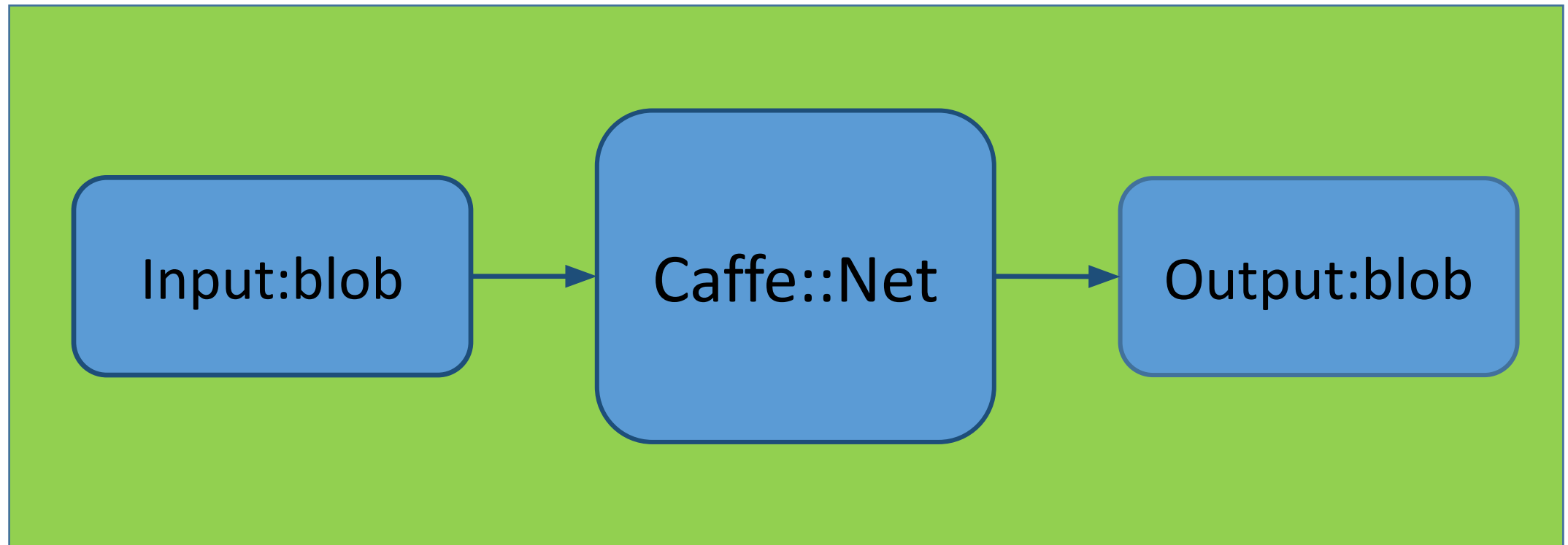


SyncedMem

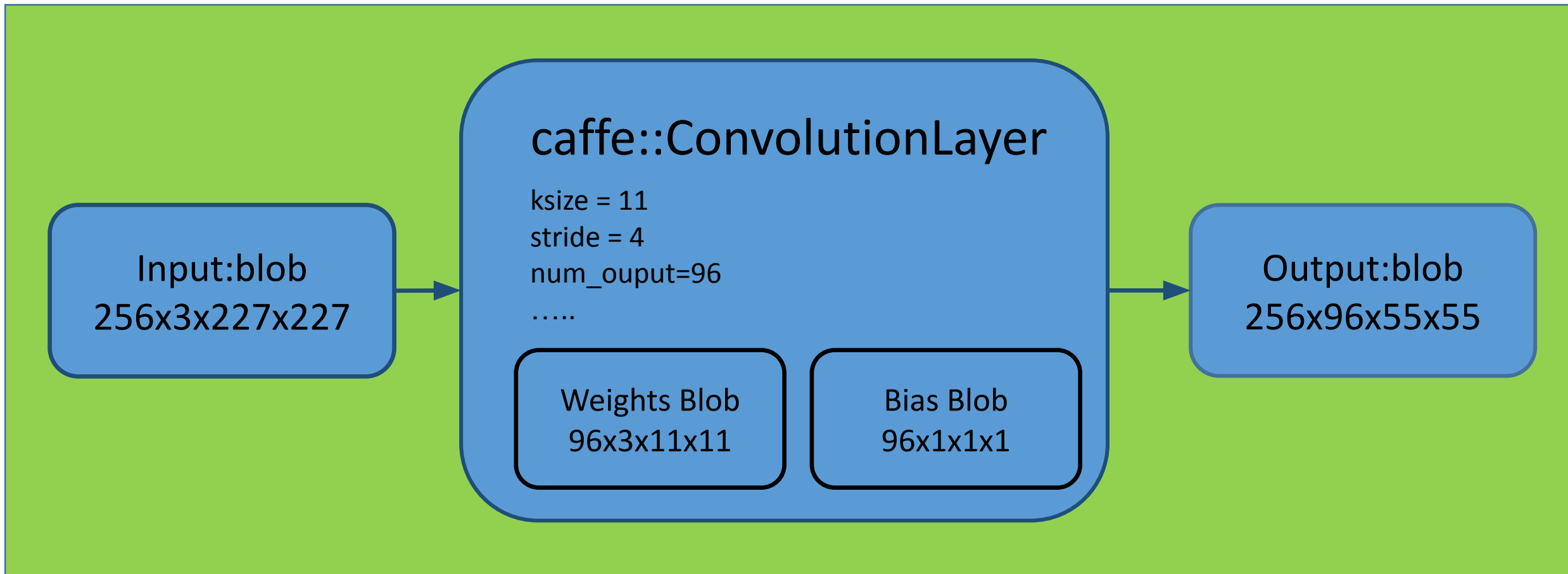
allocation + communication



Caffe Net



Caffe Net

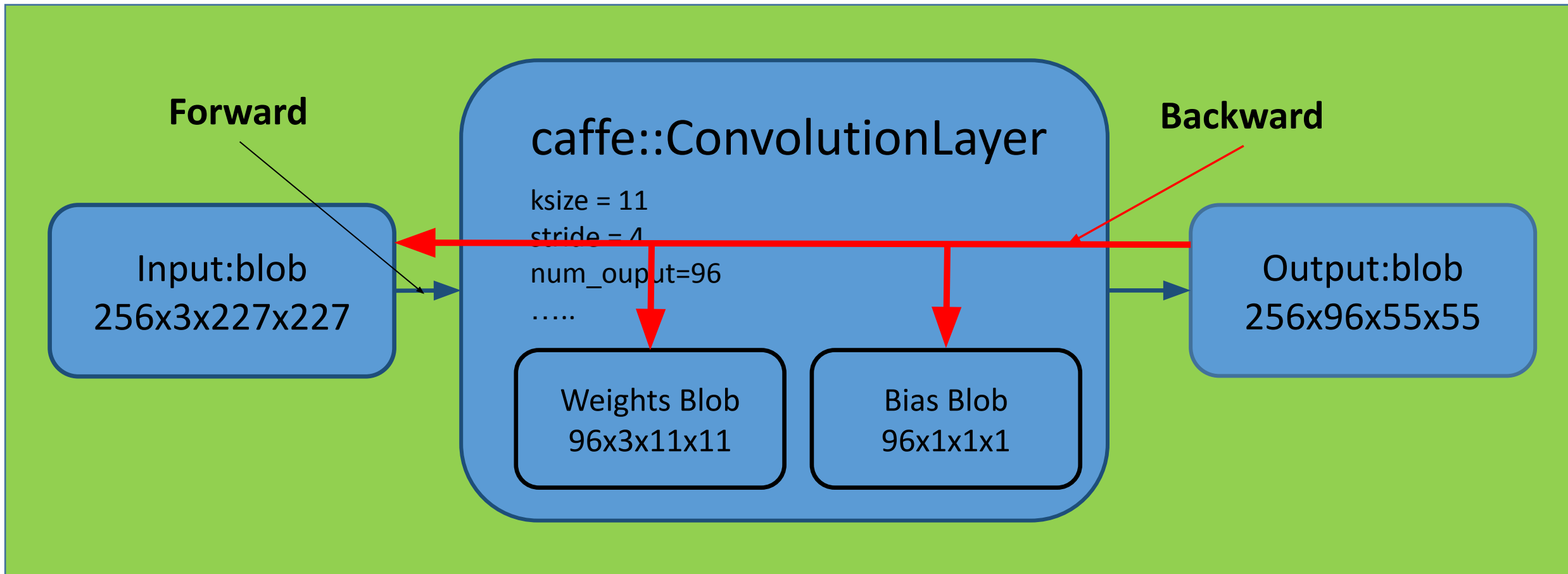


Caffe Net

Forward: given input, computes the output

Backward: given the gradient w.r.t the output, compute the gradient w.r.t the input and its internal parameters

Setup: how to initialize the layer





GPU/CPU Switch with Blob

- Use synchronized memory
- Mutable/non-mutable determines whether to copy



More about Layers

- Data layers
- Vision layers
- Common layers
- Activation/Neuron layers
- Loss layers



Data Layers

- Data enters through data layers -- they lie at the bottom of nets.
- Data can come from efficient databases (*LevelDB* or LMDB), directly from memory, or, when efficiency is not critical, from files on disk in HDF5/.mat or common image formats.
- Common input preprocessing (mean subtraction, scaling, random cropping, and mirroring) is available by specifying `TransformationParameters`.



Data Layers

- DATA
- MEMORY_DATA
- HDF5_DATA
- HDF5_OUTPUT
- IMAGE_DATA
- WINDOW_DATA
- DUMMY_DATA

Data Layers



Database

- LayerType: `DATA`
- Parameters
 - Required
 - `source`: the name of the directory containing the database
 - `batch_size`: the number of inputs to process at one time
 - Optional
 - `rand_skip`: skip up to this number of inputs at the beginning; useful for asynchronous sgd
 - `backend` [default `LEVELDB`]: choose whether to use a `LEVELDB` or `LMDB`

Data Layers



```
layers {  
  name: "mnist"  
  type: DATA  
  top: "data"  
  top: "label"  
  data_param {  
    source: "examples/mnist/mnist_train_lmdb"  
    backend: LMDB  
    batch_size: 64  
  }  
  transform_param {  
    scale: 0.00390625  
  }  
  include: { phase: TRAIN }  
}
```


Data Prep

- Inputs
- Transformations

Inputs

- Leveldb/mdb
- Raw images
- Hdf5/mat files
- Images + BB

Transformations

```
layers {  
  name: "data"  
  type: DATA  
  [...]  
  transform_param {  
    scale: 0.1  
    mean_file_size: mean.binaryproto  
    # for images in particular horizontal mirroring and random cropping  
    # can be done as simple data augmentations.  
    mirror: 1 # 1 = on, 0 = off  
    # crop a `crop_size` x `crop_size` patch:  
    # - at random during training  
    # - from the center during testing  
    crop_size: 227  
  }  
}
```



More about Layers

- Data layers
- Vision layers
- Common layers
- Activation/Neuron layers
- Loss layers



Vision layers

- Images as input and produce other images as output.
- Non-trivial height $h > 1$ and width $w > 1$.
- 2D geometry naturally lends itself to certain decisions about how to process the input.
- In particular, most of the vision layers work by applying a particular operation to some region of the input to produce a corresponding region of the output.
- In contrast, other layers (with few exceptions) ignore the spatial structure of the input, effectively treating it as “one big vector” with dimension “***chw***”.

Convolution Layer

Convolution

- LayerType: CONVOLUTION
- CPU implementation: `./src/caffe/layers/convolution_layer.cpp`
- CUDA GPU implementation: `./src/caffe/layers/convolution_layer.cu`
- Parameters (convolutionParameter convolution_param)
 - Required
 - num_output (c_o): the number of filters
 - kernel_size (or kernel_h and kernel_w): specifies height and width of each filter
 - Strongly Recommended
 - weight_filler [default type: 'constant' value: 0]
 - Optional
 - bias_term [default true]: specifies whether to learn and apply a set of additive biases to the filter outputs
 - pad (or pad_h and pad_w) [default 0]: specifies the number of pixels to (implicitly) add to each side of the input
 - stride (or stride_h and stride_w) [default 1]: specifies the intervals at which to apply the filters to the input
 - group (g) [default 1]: If $g > 1$, we restrict the connectivity of each filter to a subset of the input. Specifically, the input and output channels are separated into g groups, and the i th output group channels will be only connected to the i th input group channels.

Convolution Layer

```
layers {
  name: "conv1"
  type: CONVOLUTION
  bottom: "data"
  top: "conv1"
  blobs_lr: 1          # learning rate multiplier for the filters
  blobs_lr: 2          # learning rate multiplier for the biases
  weight_decay: 1      # weight decay multiplier for the filters
  weight_decay: 0      # weight decay multiplier for the biases
  convolution_param {
    num_output: 96      # learn 96 filters
    kernel_size: 11     # each filter is 11x11
    stride: 4           # step 4 pixels between each filter application
    weight_filler {
      type: "gaussian"  # initialize the filters from a Gaussian
      std: 0.01         # distribution with stdev 0.01 (default mean: 0)
    }
    bias_filler {
      type: "constant"  # initialize the biases to zero (0)
      value: 0
    }
  }
}
```

Pooling Layer

- LayerType: POOLING
- CPU implementation: `./src/caffe/layers/pooling_layer.cpp`
- CUDA GPU implementation: `./src/caffe/layers/pooling_layer.cu`
- Parameters (`PoolingParameter pooling_param`)
 - Required
 - `kernel_size` (or `kernel_h` and `kernel_w`): specifies height and width of each filter
 - Optional
 - `pool` [default MAX]: the pooling method. Currently MAX, AVE, or STOCHASTIC
 - `pad` (or `pad_h` and `pad_w`) [default 0]: specifies the number of pixels to (implicitly) add to each side of the input
 - `stride` (or `stride_h` and `stride_w`) [default 1]: specifies the intervals at which to apply the filters to the input

```
layers {  
  name: "pool1"  
  type: POOLING  
  bottom: "conv1"  
  top: "pool1"  
  pooling_param {  
    pool: MAX  
    kernel_size: 3 # pool over a 3x3 region  
    stride: 2      # step two pixels (in the bottom blob) between pooling regions  
  }  
}
```


Vision Layers

- Convolution
- Pooling
- Local Response Normalization (LRN)
- Im2col -- helper

More about Layers

- Data layers
- Vision layers
- Common layers
- Activation/Neuron layers
- Loss layers

Common layers

- INNER_PRODUCT $W^T x + b$ (fully conn)
- SPLIT
- FLATTEN
- CONCAT
- SLICE
- ELTWISE (element wise operations)
- ARGMAX
- SOFTMAX
- MVN (mean-variance normalization)

Common layers

```
layers {
  name: "fc8"
  type: INNER_PRODUCT
  blobs_lr: 1          # learning rate multiplier for the filters
  blobs_lr: 2          # learning rate multiplier for the biases
  weight_decay: 1      # weight decay multiplier for the filters
  weight_decay: 0      # weight decay multiplier for the biases
  inner_product_param {
    num_output: 1000
    weight_filler {
      type: "gaussian"
      std: 0.01
    }
    bias_filler {
      type: "constant"
      value: 0
    }
  }
  bottom: "fc7"
  top: "fc8"
}
```

More about Layers

- Data layers
- Vision layers
- Common layers
- Activation/Neuron layers
- Loss layers

Activation/Neuron layers

- One Input Blob
- One Output Blob
 - Both same size

Activation/Neuron layers

- RELU
- SIGMOID
- TANH
- ABSVAL
- POWER
- BNLL (binomial normal log likelihood)

Activation/Neuron layers

ReLU / Rectified-Linear and Leaky-ReLU

- LayerType: RELU
- CPU implementation: `./src/caffe/layers/relu_layer.cpp`
- CUDA GPU implementation: `./src/caffe/layers/relu_layer.cu`
- Parameters (`ReLUParameter relu_param`)
 - Optional
 - `negative_slope` [default 0]: specifies whether to leak the negative part by multiplying it with the slope value rather than setting it to 0.
- Sample (as seen in `./examples/imagenet/imagenet_train_val.prototxt`)

```
layers {  
  name: "relu1"  
  type: RELU  
  bottom: "conv1"  
  top: "conv1"  
}
```

Given an input value x , The `ReLU` layer computes the output as x if $x > 0$ and $\text{negative_slope} * x$ if $x \leq 0$. When the negative slope parameter is not set, it is equivalent to the standard ReLU function of taking $\max(x, 0)$. It also supports in-place computation, meaning that the bottom and the top blob could be the same to preserve memory consumption.

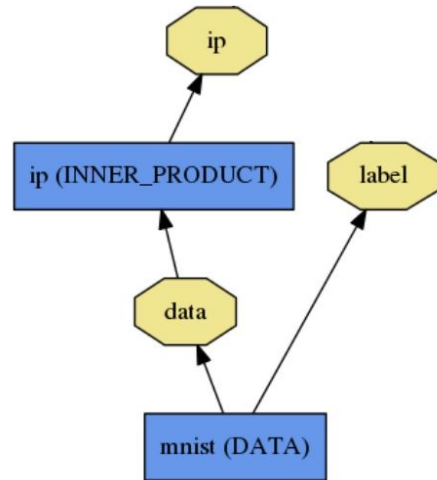
More about Layers

- Data layers
- Vision layers
- Common layers
- Activation/Neuron layers
- Loss layers

Loss

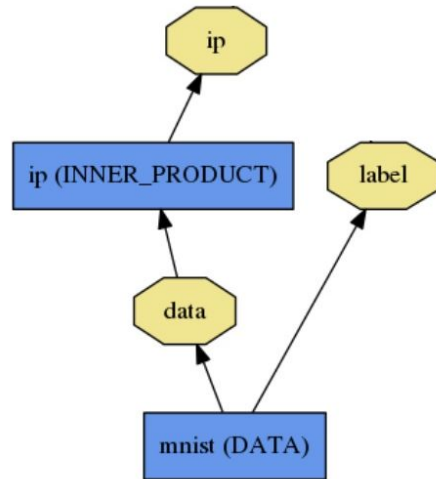


What kind of model is this?

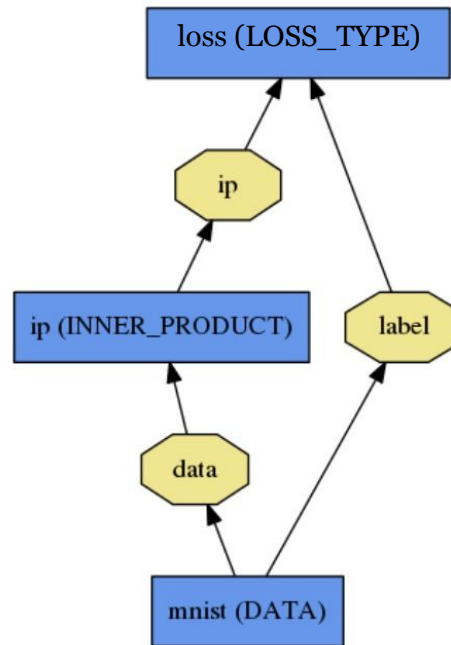


Loss

What kind of model is this?



Who knows! Need a **loss function**.



Classification

SOFTMAX_LOSS

HINGE_LOSS

Linear Regression

EUCLIDEAN_LOSS

Attributes / Multiclassification

SIGMOID_CROSS_ENTROPY_LOSS

Others...

New Task

NEW_LOSS

Loss

- **Loss function** determines the learning task
- Given data D , a Net typically minimizes:

$$L(W) = \frac{1}{|D|} \sum_i^{|D|} f_W(X^{(i)}) + \lambda r(W)$$

Data term: error
averaged over
instances

Regularization term:
penalize large weights to
improve generalization

Loss

- The data error term $f_W(X^{(i)})$ is computed by `Net::Forward`
- Loss is computed as the output of Layers
- Pick the loss to suit the task – many different losses for different needs

Loss Layers

- `SOFTMAX_LOSS`
 - `HINGE_LOSS`
 - `EUCLIDEAN_LOSS`
 - `SIGMOID_CROSS_ENTROPY_LOSS`
 - `INFOGAIN_LOSS`
-
- `ACCURACY`
 - `TOPK`

Loss Layers

- SOFTMAX_LOSS
- HINGE_LOSS
- EUCLIDEAN_LOSS
- SIGMOID_..._LOSS
- INFOGAIN_LOSS
- ACCURACY
- TOPK
- **NEW_LOSS**

Classification

Linear Regression

Attributes /

Multiclassification

Other losses

Not a loss

Softmax Loss Layer

- Multinomial logistic regression: used for predicting a single class of K mutually exclusive classes

```
layers {  
  name: "loss"  
  type: SOFTMAX_LOSS  
  bottom: "pred"  
  bottom: "label"  
  top: "loss"  
}
```

$$\hat{p}_{nk} = \frac{\exp(x_{nk})}{\sum_l \exp(x_{nl})}$$

$$E = -\frac{1}{N} \sum_{n=1}^N \log(\hat{p}_{nk})$$

Sigmoid Cross-Entropy Loss

- Binary logistic regression: used for predicting K independent probability values in [0, 1]

```
layers {  
  name: "loss"  
  type:  
    SIGMOID_CROSS_ENTROPY_LOSS  
  bottom: "pred"  
  bottom: "label"  
  top: "loss"  
}
```

$$y = (1 + \exp(-x))^{-1}$$

$$E = -\frac{1}{N} \sum_{n=1}^N [p_n \log(\hat{p}_n) + (1 - p_n) \log(1 - \hat{p}_n)]$$

Euclidean Loss

- A loss for regressing to real-valued labels $[-\infty, \infty]$

```
layers {  
  name: "loss"  
  type: EUCLIDEAN_LOSS  
  bottom: "pred"  
  bottom: "label"  
  top: "loss"  
}
```

$$E = \frac{1}{2N} \sum_{i=1}^N \|\hat{y}_i - y_i\|_2^2$$

Multiple loss layers

- Your network can contain as many loss functions as you want
- Reconstruction and Classification:

$$E = \frac{1}{2N} \sum_{i=1}^N \|\hat{y}_i - y_i\|_2^2 + -\frac{1}{N} \sum_{n=1}^N \log(\hat{p}_{nk})$$

```
layers {  
  name: "recon-loss"  
  type: EUCLIDEAN_LOSS  
  bottom: "reconstructions"  
  bottom: "data"  
  top: "recon-loss"  
}
```

```
layers {  
  name: "class-loss"  
  type: SOFTMAX_LOSS  
  bottom: "class-preds"  
  bottom: "class-labels"  
  top: "class-loss"  
}
```

Multiple loss layers

- “*_LOSS” layers have a default loss weight of 1

```
layers {  
  name: "loss"  
  type: SOFTMAX_LOSS  
  bottom: "pred"  
  bottom: "label"  
  top: "loss"  
}
```

==

```
layers {  
  name: "loss"  
  type:  
SOFTMAX_LOSS  
  bottom: "pred"  
  bottom: "label"  
  top: "loss"  
  loss_weight: 1.0  
}
```

Multiple loss layers

- Give each loss its own weight
- E.g. give higher priority to classification error

$$E = \frac{1}{2N} \sum_{i=1}^N \|\hat{y}_i - y_i\|_2^2 + 100 \left(-\frac{1}{N} \sum_{n=1}^N \log(\hat{p}_{nk}) \right)$$

```
layers {  
  name: "recon-loss"  
  type: EUCLIDEAN_LOSS  
  bottom: "reconstructions"  
  bottom: "data"  
  top: "recon-loss"  
}  
  
layers {  
  name: "class-loss"  
  type: SOFTMAX_LOSS  
  bottom: "class-preds"  
  bottom: "class-labels"  
  top: "class-loss"  
  loss_weight: 100.0  
}
```

Any layer can produce a loss!

- Just add `loss_weight: 1.0` to have a layer's output be incorporated into the loss

$$E = || \text{pred} - \text{label} ||^2 / (2N)$$

```
layers {  
  name: "loss"  
  type: EUCLIDEAN_LOSS  
  bottom: "pred"  
  bottom: "label"  
  top: "euclidean_loss"  
  loss_weight: 1.0  
}
```

$$\text{diff} = \text{pred} - \text{label}$$

```
layers {  
  name: "diff"  
  type: ELTWISE  
  bottom: "pred"  
  bottom: "label"  
  top: "diff"  
  eltwise_param {  
    op: SUM  
    coeff: 1  
    coeff: -1  
  }  
}
```

$$E = || \text{diff} ||^2 / (2N)$$

```
layers {  
  name: "loss"  
  type: POWER  
  bottom: "diff"  
  top: "euclidean_loss"  
  power_param {  
    power: 2  
  }  
  # = 1/(2N)  
  loss_weight: 0.0078125  
}
```

Initialization

- Gaussian
- Xavier





Solving: Training a Net

Optimization like model definition is configuration.

```
train_net: "lenet_train.prototxt"
```

```
base_lr: 0.01
```

```
momentum: 0.9
```

```
weight_decay: 0.0005
```

```
max_iter: 10000
```

```
snapshot_prefix: "lenet_snapshot"
```

All you need to run things
on the GPU.



```
> caffe train -solver lenet_solver.prototxt -gpu 0
```

Stochastic Gradient Descent (SGD) + momentum •

Adaptive Gradient (ADAGRAD) • Nesterov's Accelerated Gradient (NAG)


```
# The train/test net protocol buffer definition
net: "logreg_train.prototxt"
# test_iter specifies how many forward passes the test should carry out.
# In the case of MNIST, we have test batch size 100 and 100 test iterations,
# covering the full 10,000 testing images.
test_iter: 100
# Carry out testing every 500 training iterations.
test_interval: 500
# The base learning rate, momentum and the weight decay of the network.
base_lr: 0.01
momentum: 0.9
weight_decay: 0.0005
# The learning rate policy
lr_policy: "inv"
gamma: 0.0001
power: 0.75
# Display every 100 iterations
display: 100
# The maximum number of iterations
max_iter: 10000
# snapshot intermediate results
snapshot: 5000
snapshot_prefix: "examples/mnist/lenet"
# solver mode: CPU or GPU
solver_mode: GPU
```



Solver

- **Solver** optimizes the network weights W to minimize the loss $L(W)$ over the data D

$$L(W) = \frac{1}{|D|} \sum_i^{|D|} f_W(X^{(i)}) + \lambda r(W)$$

- Coordinates forward / backward, weight updates, and scoring.



Solver

- Computes parameter update ΔW , formed from
- The stochastic error gradient Δf_W
- The regularization gradient $\Delta r(W)$
- Particulars to each solving method

$$L(W) \approx \frac{1}{N} \sum_i^N f_W(X^{(i)}) + \lambda r(W)$$

SGD Solver

- Stochastic gradient descent, with momentum
- `solver_type: SGD`

$$V_{t+1} = \beta V_t - \lambda \nabla L(W_t)$$

$$W_{t+1} = W_t + V_{t+1}$$

SGD Solver

- “AlexNet” [1] training strategy:
 - Use momentum 0.9
 - Initialize learning rate at 0.01
 - Periodically drop learning rate by a factor of 10
- Just a few lines of Caffe solver specification:

```
base_lr: 0.01
lr_policy: "step"
gamma: 0.1
stepsize: 100000
max_iter: 350000
momentum: 0.9
```

NAG Solver

- Nesterov's accelerated gradient
- `solver_type: NESTEROV`
- Proven to have optimal convergence rate $\Theta(1/t^2)$ for convex problems

$$V_{t+1} = \beta V_t - \lambda \nabla L(W_t + \beta V_t)$$

$$W_{t+1} = W_t + V_{t+1}$$

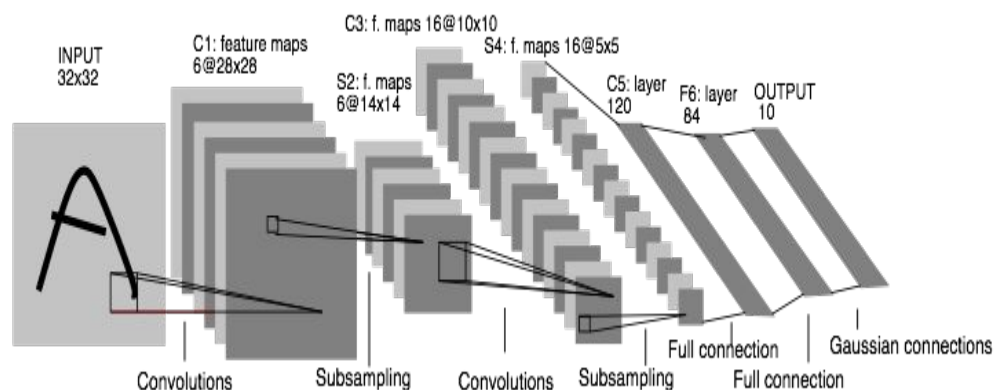
AdaGrad Solver

- Adaptive gradient (Duchi et al. [1])
- `solver_type: ADAGRAD`
- Attempts to automatically scale gradients based on historical gradients

$$(W_{t+1})_i = (W_t)_i - \alpha \frac{(\nabla L(W_t))_i}{\sqrt{\sum_{t'=1}^t (\nabla L(W_{t'}))_i^2}}$$

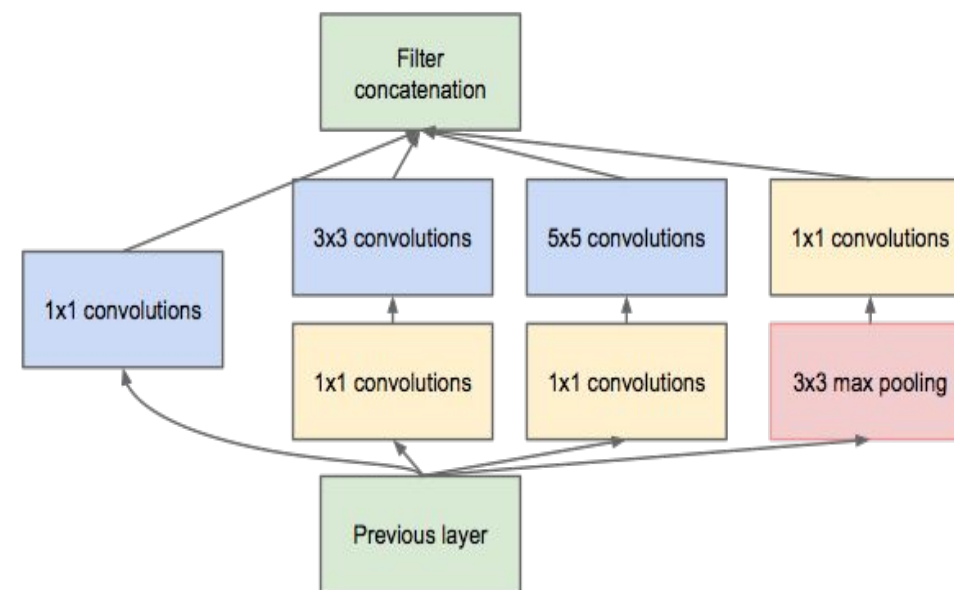
DAG

Many current deep models have linear structure



but Caffe nets can have any directed acyclic graph (DAG) structure.

Define bottoms and tops
and Caffe will connect the net.



GoogLeNet Inception Module



Weight sharing

- Parameters can be shared and reused across Layers throughout the Net
- Applications:
 - Convolution at multiple scales / pyramids
 - Recurrent Neural Networks (RNNs)
 - Siamese nets for distance learning

Weight sharing

- Just give the parameter blobs explicit names using the `param` field
- Layers specifying the same `param` name will share that parameter, accumulating gradients accordingly

```
layers: {  
  name: 'innerproduct1'  
  type: INNER_PRODUCT  
  inner_product_param {  
    num_output: 10  
    bias_term: false  
    weight_filler {  
      type: 'gaussian'  
      std: 10  
    }  
  }  
  param: 'sharedweights'  
  bottom: 'data'  
  top: 'innerproduct1'  
}  
layers: {  
  name: 'innerproduct2'  
  type: INNER_PRODUCT  
  inner_product_param {  
    num_output: 10  
    bias_term: false  
  }  
  param: 'sharedweights'  
  bottom: 'data'  
  top: 'innerproduct2'  
}
```



CMD



- `$> Caffe --params`

```
# train LeNet
caffe train -solver examples/mnist/lenet_solver.prototxt
# train on GPU 2
caffe train -solver examples/mnist/lenet_solver.prototxt -gpu 2
# resume training from the half-way point snapshot
caffe train -solver examples/mnist/lenet_solver.prototxt -snapshot
examples/mnist/lenet_iter_5000.solverstate
```

CMD



```
# fine-tune CaffeNet model weights for pascal
caffe train \
-solver examples/finetune_pascal_detection/pascal_finetune_solver.prototxt -
weights models/bvlc_reference_caffenet/bvlc_reference_caffenet.caffemodel
```

```
# score the learned LeNet model on the validation set as defined in the model
architecture lenet_train_test.prototxt
caffe test -model examples/mnist/lenet_train_test.prototxt -weights
examples/mnist/lenet_iter_10000 -gpu 0 -iterations 100
```



Assignment: New Layer

Write a new Layer

- Update `caffe.proto`
 - Add in layer list (optionally add `new_layer_param`)
- Add in `layer_factory.cpp`
- Declare in either `vision_layers.hpp`, `data_layers.hpp`, `common_layers.hpp`, `neuron_layers.hpp` or `loss_layers.hpp`
- Do nearest neighbour copy of existing layer (cpp+cu)
- Write `SetUp`, `ForwardPass` & `BackwardPass` (cpp+cu)
- Done!!



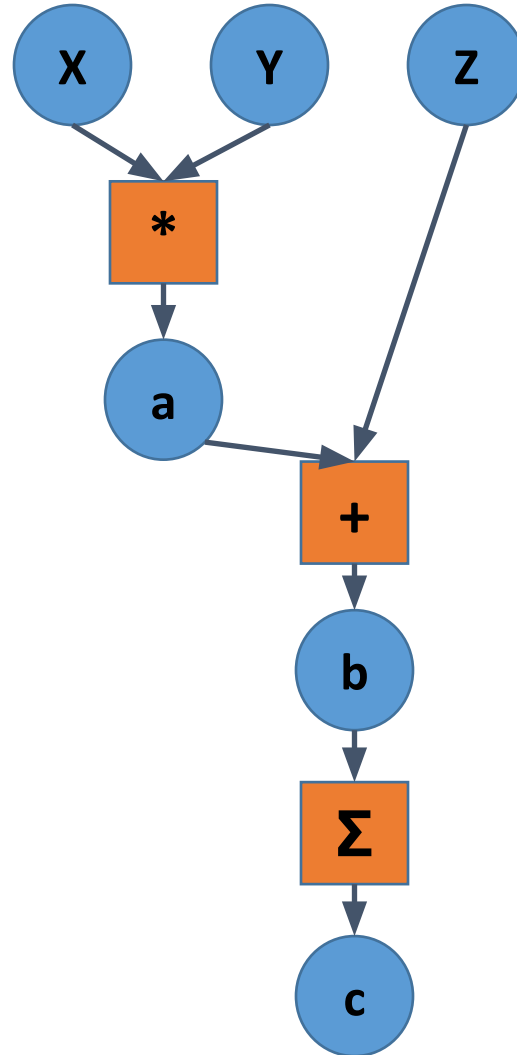
Deep Learning Framework

Deep learning and Its Applications in Python

The points of deep learning frameworks

- (1) Easily build big computational graphs
- (2) Easily compute gradients in computational graphs
- (3) Run it all efficiently on GPU (wrap cuDNN, cuBLAS, etc)

Computational Graphs



Computational Graphs

```
import numpy as np
np.random.seed(0)
```

```
N, D = 3, 4
```

```
x = np.random.randn(N, D)
```

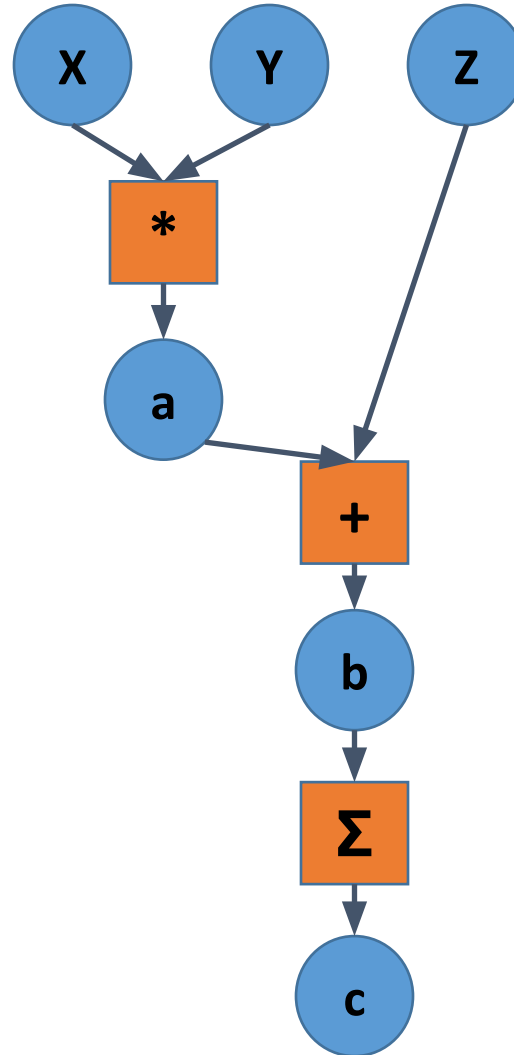
```
y = np.random.randn(N, D)
```

```
z = np.random.randn(N, D)
```

```
a = x * y
```

```
b = a + z
```

```
c = np.sum(b)
```



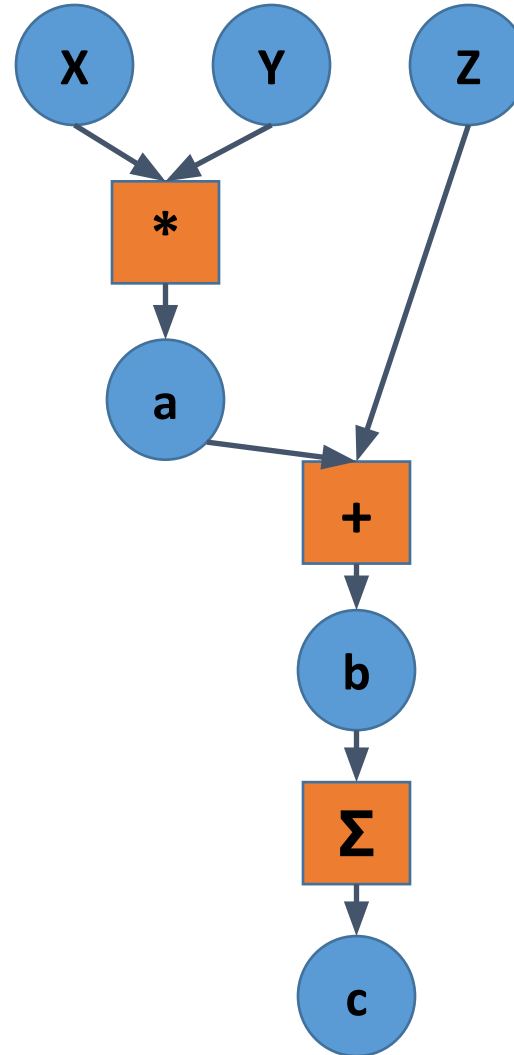
Computational Graphs

```
import numpy as np
np.random.seed(0)

N, D = 3, 4
x = np.random.randn(N, D)
y = np.random.randn(N, D)
z = np.random.randn(N, D)

a = x * y
b = a + z
c = np.sum(b)

grad_c = 1.0
grad_b = grad_c * np.ones((N, D))
grad_a = grad_b.copy()
grad_z = grad_b.copy()
grad_x = grad_a * x
grad_y = grad_a * y
```



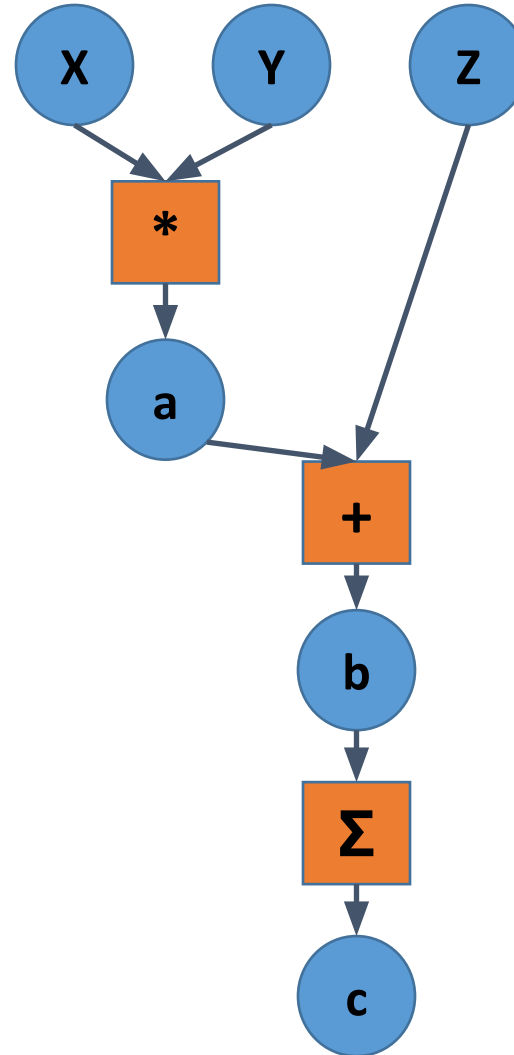
Computational Graphs

```
import numpy as np
np.random.seed(0)

N, D = 3, 4
x = np.random.randn(N, D)
y = np.random.randn(N, D)
z = np.random.randn(N, D)

a = x * y
b = a + z
c = np.sum(b)

grad_c = 1.0
grad_b = grad_c * np.ones((N, D))
grad_a = grad_b.copy()
grad_z = grad_b.copy()
grad_x = grad_a * x
grad_y = grad_a * y
```



Problems:

- Can't run on GPU
- Have to compute our own gradients



TensorFlow Solution

Deep learning and Its Applications in Python

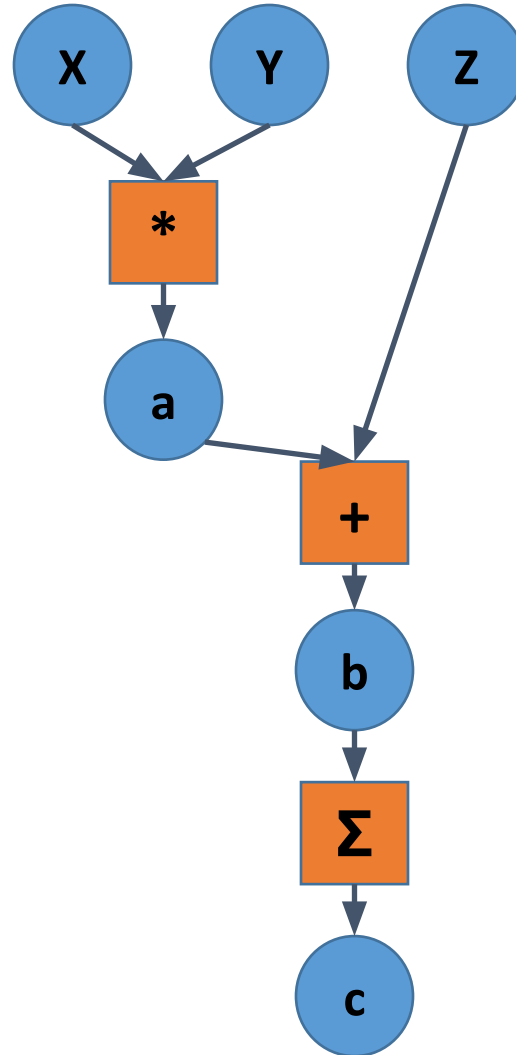
Computational Graphs

```
import numpy as np
np.random.seed(0)

N, D = 3, 4
x = np.random.randn(N, D)
y = np.random.randn(N, D)
z = np.random.randn(N, D)

a = x * y
b = a + z
c = np.sum(b)

grad_c = 1.0
grad_b = grad_c * np.ones((N, D))
grad_a = grad_b.copy()
grad_z = grad_b.copy()
grad_x = grad_a * x
grad_y = grad_a * y
```



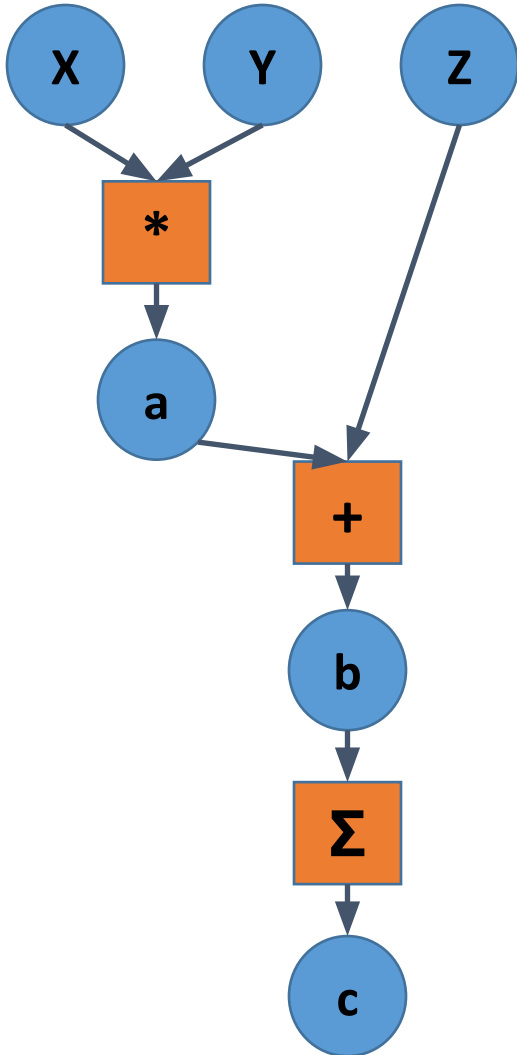
```
import numpy as np
np.random.seed(0)
import tensorflow as tf

N, D = 3, 4
x = tf.placeholder(tf.float32)
y = tf.placeholder(tf.float32)
z = tf.placeholder(tf.float32)

a = x * y
b = a + z
c = tf.reduce_sum(b)
grad_x, grad_y, grad_z = tf.gradients(c, [x, y, z])

with tf.Session() as sess:
    values = {
        x: np.random.randn(N, D),
        y: np.random.randn(N, D),
        z: np.random.randn(N, D)
    }
    out = sess.run([c, grad_x, grad_y, grad_z], feed_dict = values)
    c_val, grad_x_val, grad_y_val, grad_z_val = out
```

Computational Graphs



Create forward
computational graph

```
import numpy as np
np.random.seed(0)
import tensorflow as tf

N, D = 3, 4

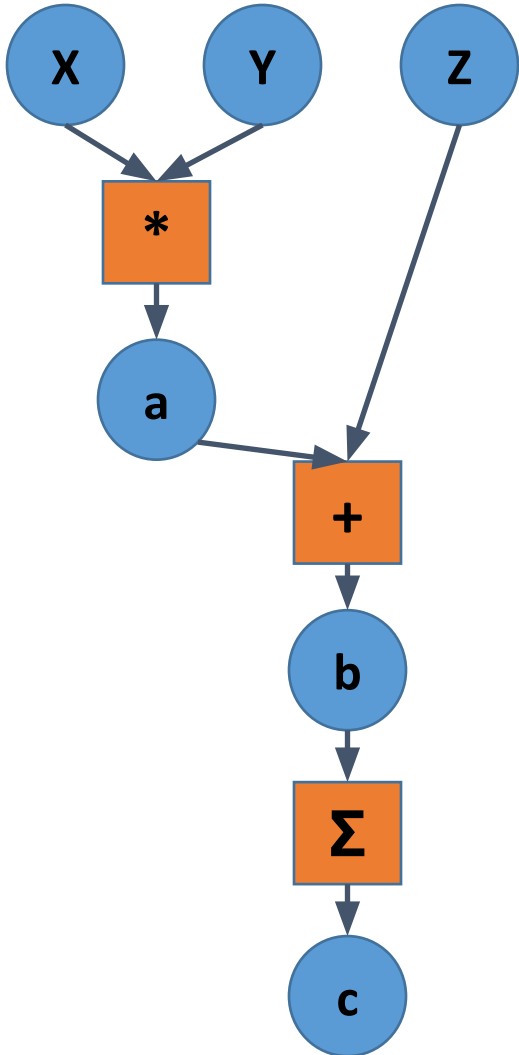
x = tf.placeholder(tf.float32)
y = tf.placeholder(tf.float32)
z = tf.placeholder(tf.float32)

a = x * y
b = a + z
c = tf.reduce_sum(b)

grad_x, grad_y, grad_z = tf.gradients(c, [x, y, z])

with tf.Session() as sess:
    values = {
        x: np.random.randn(N, D),
        y: np.random.randn(N, D),
        z: np.random.randn(N, D)
    }
    out = sess.run([c, grad_x, grad_y, grad_z], feed_dict = values)
    c_val, grad_x_val, grad_y_val, grad_z_val = out
```

Computational Graphs



Ask Tensorflow to
compute Gradients

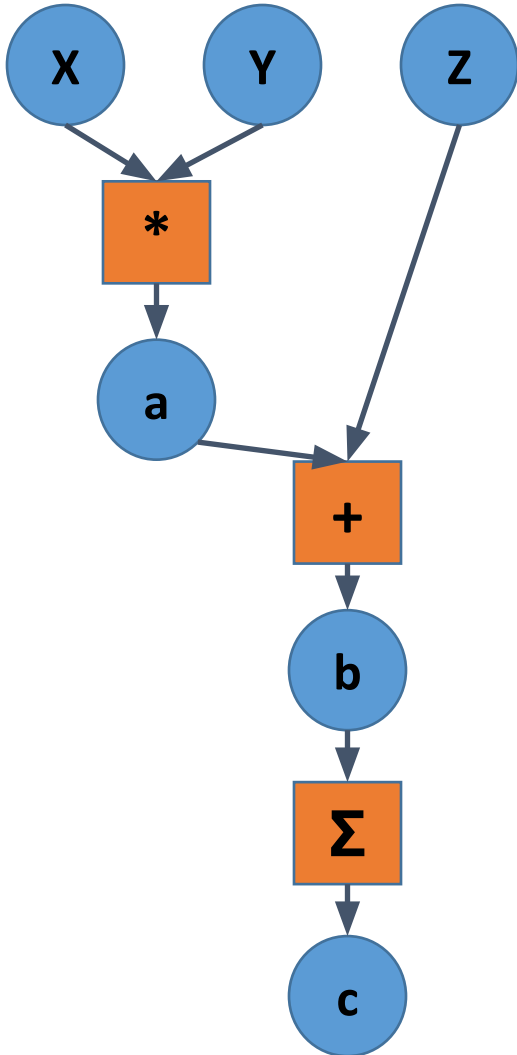
```
import numpy as np
np.random.seed(0)
import tensorflow as tf

N, D = 3, 4
x = tf.placeholder(tf.float32)
y = tf.placeholder(tf.float32)
z = tf.placeholder(tf.float32)

a = x * y
b = a + z
c = tf.reduce_sum(b)
grad_x, grad_y, grad_z = tf.gradients(c, [x, y, z])

with tf.Session() as sess:
    values = {
        x: np.random.randn(N, D),
        y: np.random.randn(N, D),
        z: np.random.randn(N, D)
    }
    out = sess.run([c, grad_x, grad_y, grad_z], feed_dict = values)
    c_val, grad_x_val, grad_y_val, grad_z_val = out
```

Computational Graphs



Tell Tensorflow to
run on **CPU**

```
import numpy as np
np.random.seed(0)
import tensorflow as tf

N, D = 3, 4

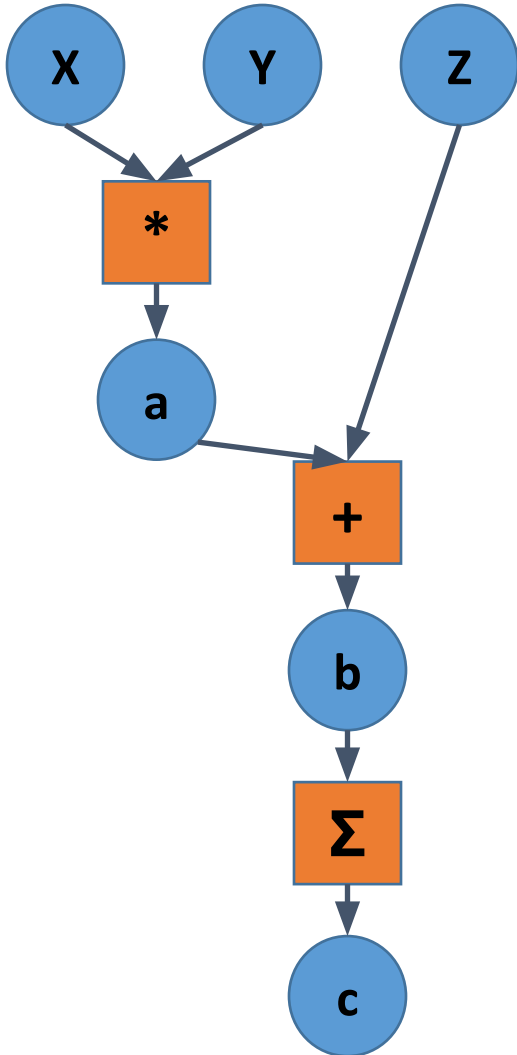
with tf.device('/cpu:0'):
    x = tf.placeholder(tf.float32)
    y = tf.placeholder(tf.float32)
    z = tf.placeholder(tf.float32)

    a = x * y
    b = a + z
    c = tf.reduce_sum(b)

grad_x, grad_y, grad_z = tf.gradients(c, [x, y, z])

with tf.Session() as sess:
    values = {
        x: np.random.randn(N, D),
        y: np.random.randn(N, D),
        z: np.random.randn(N, D)
    }
    out = sess.run([c, grad_x, grad_y, grad_z], feed_dict = values)
    c_val, grad_x_val, grad_y_val, grad_z_val = out
```


Computational Graphs



Tell Tensorflow to
run on **GPU**

```
import numpy as np
np.random.seed(0)
import tensorflow as tf

N, D = 3, 4

with tf.device('/gpu:0'):
    x = tf.placeholder(tf.float32)
    y = tf.placeholder(tf.float32)
    z = tf.placeholder(tf.float32)

    a = x * y
    b = a + z
    c = tf.reduce_sum(b)

grad_x, grad_y, grad_z = tf.gradients(c, [x, y, z])

with tf.Session() as sess:
    values = {
        x: np.random.randn(N, D),
        y: np.random.randn(N, D),
        z: np.random.randn(N, D)
    }
    out = sess.run([c, grad_x, grad_y, grad_z], feed_dict = values)
    c_val, grad_x_val, grad_y_val, grad_z_val = out
```



PyTorch Solution

Deep learning and Its Applications in Python

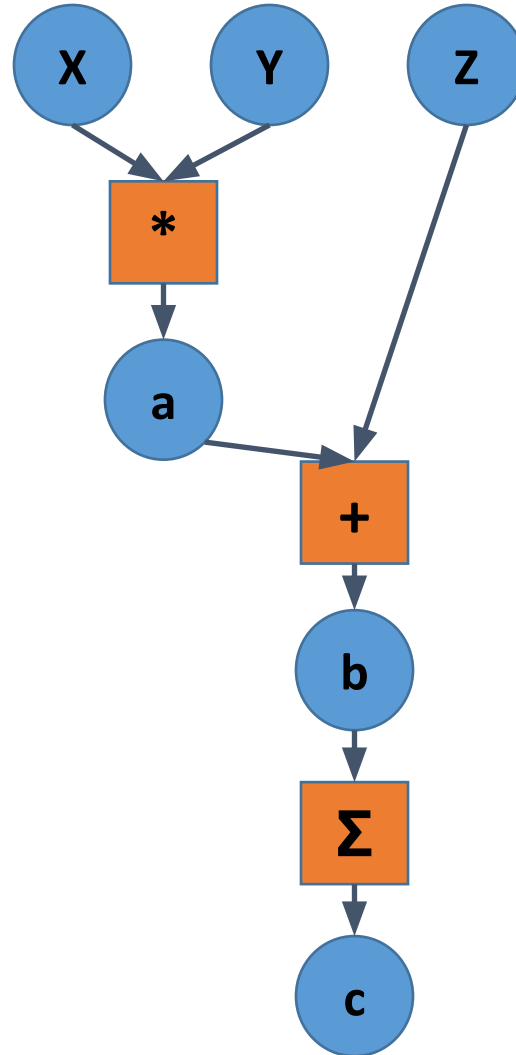
Computational Graphs

```
import numpy as np
np.random.seed(0)

N, D = 3, 4
x = np.random.randn(N, D)
y = np.random.randn(N, D)
z = np.random.randn(N, D)

a = x * y
b = a + z
c = np.sum(b)

grad_c = 1.0
grad_b = grad_c * np.ones((N, D))
grad_a = grad_b.copy()
grad_z = grad_b.copy()
grad_x = grad_a * x
grad_y = grad_a * y
```



```
import torch
from torch.autograd import Variable

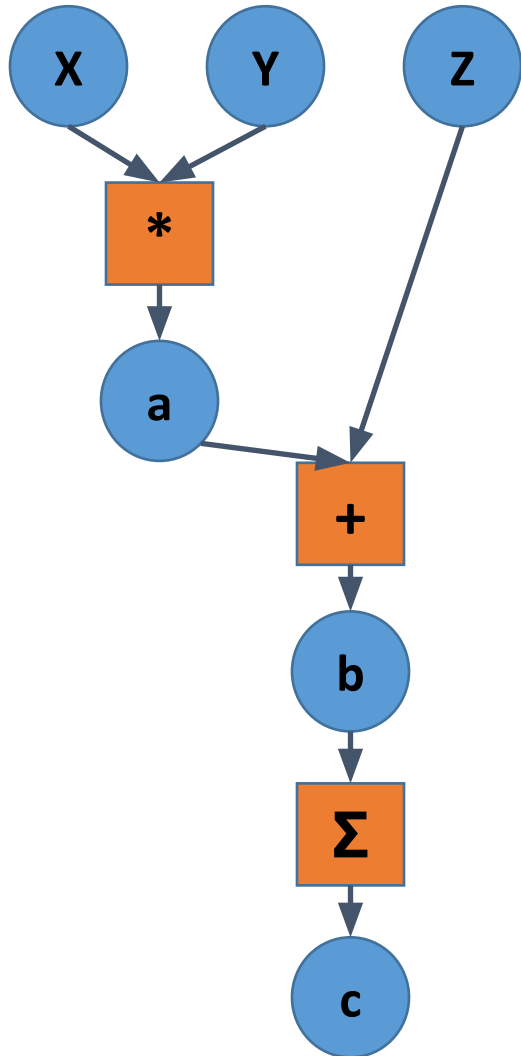
N, D = 3, 4
x = Variable(torch.randn(N, D),
              requires_grad=True)
y = Variable(torch.randn(N, D),
              requires_grad=True)
z = Variable(torch.randn(N, D),
              requires_grad=True)

a = x * y
b = a + z
c = torch.sum(b)

c.backward()

print(x.grad.data)
print(y.grad.data)
print(z.grad.data)
```

Computational Graphs



Define Variable and
start building a
computational graph

```
import torch
from torch.autograd import Variable

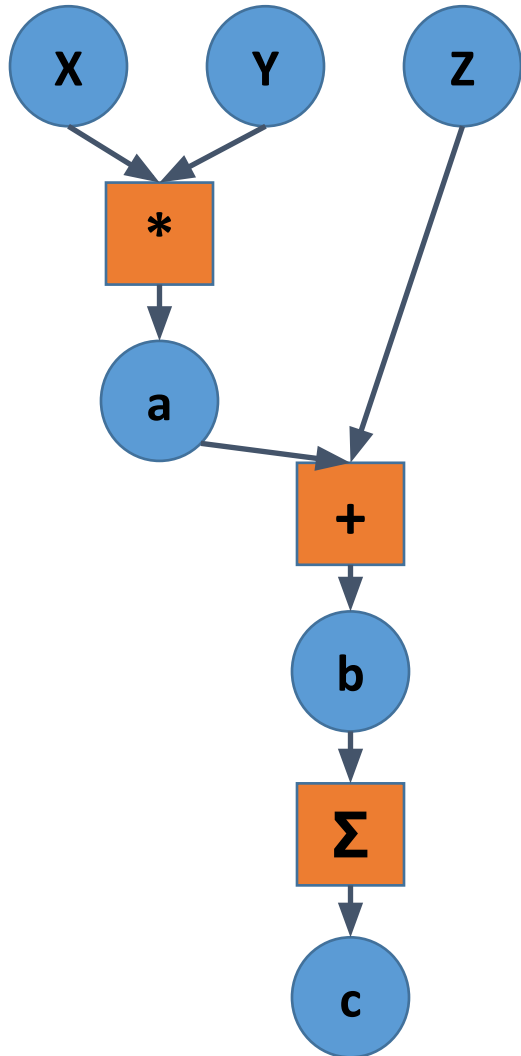
N, D = 3, 4
x = Variable(torch.randn(N, D),
              requires_grad=True)
y = Variable(torch.randn(N, D),
              requires_grad=True)
z = Variable(torch.randn(N, D),
              requires_grad=True)

a = x * y
b = a + z
c = torch.sum(b)

c.backward()

print(x.grad.data)
print(y.grad.data)
print(z.grad.data)
```

Computational Graphs



Fordwss looks just like numpy

```
import torch
from torch.autograd import Variable
```

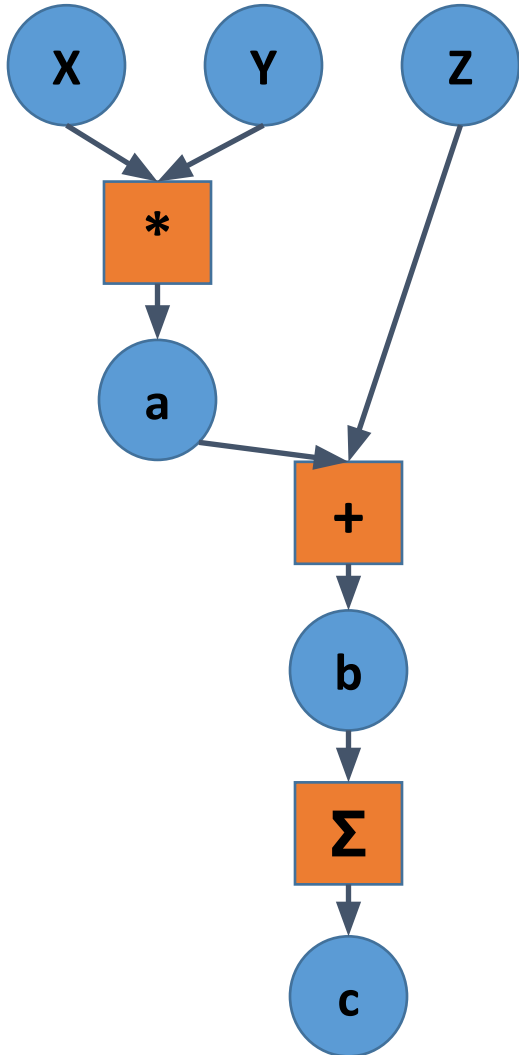
```
N, D = 3, 4
x = Variable(torch.randn(N, D),
              requires_grad=True)
y = Variable(torch.randn(N, D),
              requires_grad=True)
z = Variable(torch.randn(N, D),
              requires_grad=True)
```

```
a = x * y
b = a + z
c = torch.sum(b)
```

```
c.backward()
```

```
print(x.grad.data)
print(y.grad.data)
print(z.grad.data)
```

Computational Graphs



Call backward()
compute all gradient

```
import torch
from torch.autograd import Variable

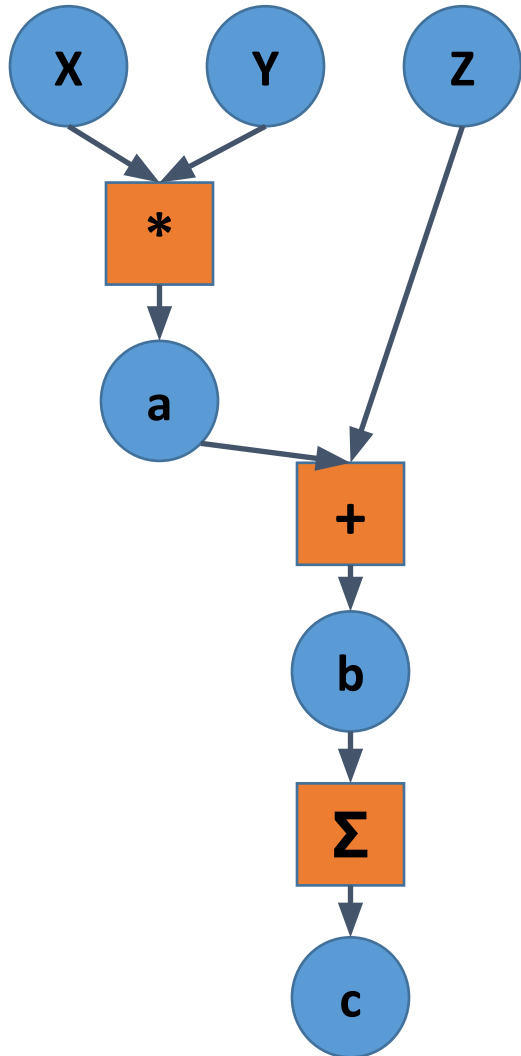
N, D = 3, 4
x = Variable(torch.randn(N, D),
              requires_grad=True)
y = Variable(torch.randn(N, D),
              requires_grad=True)
z = Variable(torch.randn(N, D),
              requires_grad=True)

a = x * y
b = a + z
c = torch.sum(b)

c.backward()

print(x.grad.data)
print(y.grad.data)
print(z.grad.data)
```

Computational Graphs



Run on GPU by
casting to cuda

```
import torch
from torch.autograd import Variable

N, D = 3, 4
x = Variable(torch.randn(N, D).cuda(),
              requires_grad=True)
y = Variable(torch.randn(N, D).cuda(),
              requires_grad=True)
z = Variable(torch.randn(N, D).cuda(),
              requires_grad=True)

a = x * y
b = a + z
c = torch.sum(b)

c.backward()

print(x.grad.data)
print(y.grad.data)
print(z.grad.data)
```

Computational Graphs

```
import numpy as np
np.random.seed(0)

N, D = 3, 4
x = np.random.randn(N,D)
y = np.random.randn(N,D)
z = np.random.randn(N,D)

a = x * y
b = a + z
c = np.sum(b)

grad_c = 1.0
grad_b = grad_c *
np.ones( (N,D) )
grad_a = grad_b.copy()
grad_z = grad_b.copy()
grad_x = grad_a * x
grad_y = grad_a * y
```

Numpy

```
import numpy as np
np.random.seed(0)
import tensorflow as tf

N, D = 3, 4
x = tf.placeholder(tf.float32)
y = tf.placeholder(tf.float32)
z = tf.placeholder(tf.float32)

a = x * y
b = a + z
c = tf.reduce_sum(b)
grad_x, grad_y, grad_z =
tf.gradients(c, [x, y, z])
with tf.Session() as sess:
    values = {
        x: np.random.randn(N,D),
        y: np.random.randn(N,D),
        z: np.random.randn(N,D)
    }
    out = sess.run([c, grad_x, grad_y,
grad_z], feed_dict = values)
    c_val, grad_x_val, grad_y_val,
grad_z_val = out
```

TensorFlow

```
import torch
from torch.autograd import
Variable

N, D = 3, 4
x = Variable(torch.randn(N,D),
requires_grad=True)
y = Variable(torch.randn(N,D),
requires_grad=True)
z = Variable(torch.randn(N,D),
requires_grad=True)

a = x * y
b = a + z
c = torch.sum(b)

c.backward()

print(x.grad.data)
print(y.grad.data)
print(z.grad.data)
```

Torch



TensorFlow

Deep learning and Its Applications in Python

TensorFlow: Neural Net

Running example:
Train a two-layer
ReLU network on
random data
with L2 loss

```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N,D))
y = tf.placeholder(tf.float32, shape=(N,D))
w1 = tf.placeholder(tf.float32, shape=(D,H))
w2 = tf.placeholder(tf.float32, shape=(H,D))

h = tf.maximun(tf.matmul(x, w1), 0)
y_pred = tf.matmul(h,w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff**2, axis=1))
grad_w1, grad_w2 = tf.gradients(loss, [w1, w2])
with tf.Session() as sess
    values = {
        x: np.random.randn(N,D)
        w1: np.random.randn(D,H)
        w2: np.random.randn(H,D)
        y: np.random.randn(N,D)
    }

    out = sess.run([loss, grad_w1, grad_w2], feed_dict = values)
    loss_val, grad_w1_val, grad_w2_val = out
```

TensorFlow: Neural Net

First define
computational graph



```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N,D))
y = tf.placeholder(tf.float32, shape=(N,D))
w1 = tf.placeholder(tf.float32, shape=(D,H))
w2 = tf.placeholder(tf.float32, shape=(H,D))

h = tf.maximun(tf.matmul(x, w1), 0)
y_pred = tf.matmul(h,w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff**2, axis=1))
grad w1, grad w2 = tf.gradients(loss, [w1, w2])
```

The run the graph
many times



```
with tf.Session() as sess
    values = {
        x: np.random.randn(N,D)
        w1: np.random.randn(D,H)
        w2: np.random.randn(H,D)
        y: np.random.randn(N,D)
    }

    out = sess.run([loss, grad_w1, grad_w2], feed_dict = values)
    loss_val, grad_w1_val, grad_w2_val = out
```

TensorFlow: Neural Net

Create **placeholders**
for input x, weights
w1 and w2, and
targets y

```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N,D))
y = tf.placeholder(tf.float32, shape=(N,D))
w1 = tf.placeholder(tf.float32, shape=(D,H))
w2 = tf.placeholder(tf.float32, shape=(H,D))

h = tf.maximun(tf.matmul(x, w1), 0)
y_pred = tf.matmul(h,w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff**2, axis=1))
grad_w1, grad_w2 = tf.gradients(loss, [w1, w2])
with tf.Session() as sess
    values = {
        x: np.random.randn(N,D)
        w1: np.random.randn(D,H)
        w2: np.random.randn(H,D)
        y: np.random.randn(N,D)
    }

    out = sess.run([loss, grad_w1, grad_w2], feed_dict = values)
    loss_val, grad_w1_val, grad_w2_val = out
```

TensorFlow: Neural Net

Forward pass: compute prediction for y and loss (L2 distance between y and y_pred)

No computation happens here
– just building the graph!

```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N,D))
y = tf.placeholder(tf.float32, shape=(N,D))
w1 = tf.placeholder(tf.float32, shape=(D,H))
w2 = tf.placeholder(tf.float32, shape=(H,D))

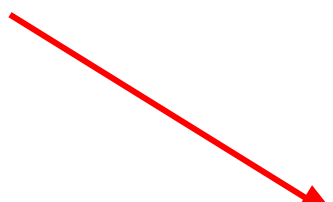
h = tf.maximun(tf.matmul(x, w1), 0)
y_pred = tf.matmul(h,w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff**2, axis=1))
grad_w1, grad_w2 = tf.gradients(loss, [w1, w2])

with tf.Session() as sess
    values = {
        x: np.random.randn(N,D)
        w1: np.random.randn(D,H)
        w2: np.random.randn(H,D)
        y: np.random.randn(N,D)
    }

    out = sess.run([loss, grad_w1, grad_w2], feed_dict = values)
    loss_val, grad_w1_val, grad_w2_val = out
```

TensorFlow: Neural Net

Tell TensorFlow to
compute loss of
gradient with respect to
w1 and w2.



No computation happens here
– just building the graph!

```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N,D))
y = tf.placeholder(tf.float32, shape=(N,D))
w1 = tf.placeholder(tf.float32, shape=(D,H))
w2 = tf.placeholder(tf.float32, shape=(H,D))

h = tf.maximun(tf.matmul(x, w1), 0)
y_pred = tf.matmul(h,w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff**2, axis=1))
grad_w1, grad_w2 = tf.gradients(loss, [w1, w2])
with tf.Session() as sess
    values = {
        x: np.random.randn(N,D)
        w1: np.random.randn(D,H)
        w2: np.random.randn(H,D)
        y: np.random.randn(N,D)
    }

    out = sess.run([loss, grad_w1, grad_w2], feed_dict = values)
    loss_val, grad_w1_val, grad_w2_val = out
```

TensorFlow: Neural Net

Now done building
our graph, so we
enter a **session** so
we can actually run
the graph

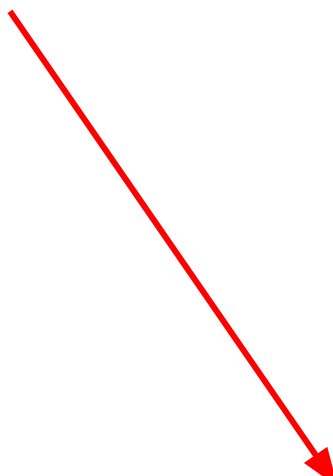
```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N,D))
y = tf.placeholder(tf.float32, shape=(N,D))
w1 = tf.placeholder(tf.float32, shape=(D,H))
w2 = tf.placeholder(tf.float32, shape=(H,D))

h = tf.maximun(tf.matmul(x, w1), 0)
y_pred = tf.matmul(h,w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff**2, axis=1))
grad_w1, grad_w2 = tf.gradients(loss, [w1, w2])
with tf.Session() as sess
    values = {
        x: np.random.randn(N,D)
        w1: np.random.randn(D,H)
        w2: np.random.randn(H,D)
        y: np.random.randn(N,D)
    }

    out = sess.run([loss, grad_w1, grad_w2], feed_dict = values)
    loss_val, grad_w1_val, grad_w2_val = out
```

TensorFlow: Neural Net

Create numpy arrays
that will fill in the
placeholders above

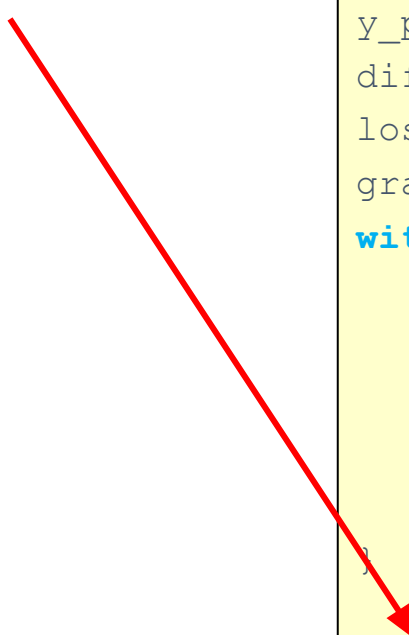


```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N,D))
y = tf.placeholder(tf.float32, shape=(N,D))
w1 = tf.placeholder(tf.float32, shape=(D,H))
w2 = tf.placeholder(tf.float32, shape=(H,D))

h = tf.maximun(tf.matmul(x, w1), 0)
y_pred = tf.matmul(h,w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff**2, axis=1))
grad_w1, grad_w2 = tf.gradients(loss, [w1, w2])
with tf.Session() as sess
    values = {
        x: np.random.randn(N,D)
        w1: np.random.randn(D,H)
        w2: np.random.randn(H,D)
        y: np.random.randn(N,D)
    }
    out = sess.run([loss, grad_w1, grad_w2], feed_dict = values)
    loss_val, grad_w1_val, grad_w2_val = out
```


TensorFlow: Neural Net

Run the graph: feed in the numpy arrays for x, y, w1, and w2; get numpy arrays for loss, grad_w1, and grad_w2

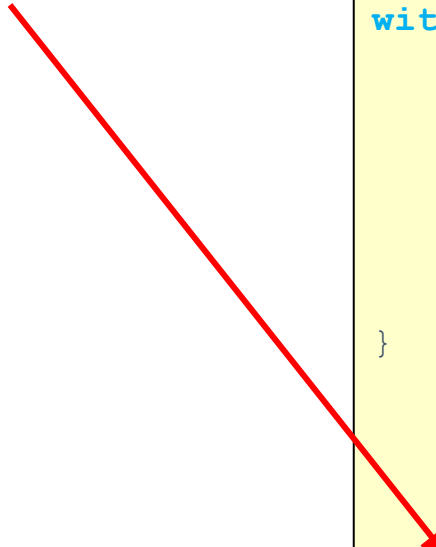


```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N,D))
y = tf.placeholder(tf.float32, shape=(N,D))
w1 = tf.placeholder(tf.float32, shape=(D,H))
w2 = tf.placeholder(tf.float32, shape=(H,D))

h = tf.maximun(tf.matmul(x, w1), 0)
y_pred = tf.matmul(h,w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff**2, axis=1))
grad_w1, grad_w2 = tf.gradients(loss, [w1, w2])
with tf.Session() as sess
    values = {
        x: np.random.randn(N,D)
        w1: np.random.randn(D,H)
        w2: np.random.randn(H,D)
        y: np.random.randn(N,D)
    }
    out = sess.run([loss, grad_w1, grad_w2], feed_dict = values)
    loss_val, grad_w1_val, grad_w2_val = out
```

TensorFlow: Neural Net

Train the network: Run the graph over and over, use gradient to update weights



```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N,D))
y = tf.placeholder(tf.float32, shape=(N,D))
w1 = tf.placeholder(tf.float32, shape=(D,H))
w2 = tf.placeholder(tf.float32, shape=(H,D))

h = tf.matmul(x, w1)
y_pred = tf.matmul(h, w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff**2, axis=1))
grad_w1, grad_w2 = tf.gradients(loss, [w1, w2])
with tf.Session() as sess
    values = {
        x: np.random.randn(N,D)
        w1: np.random.randn(D,H)
        w2: np.random.randn(H,D)
        y: np.random.randn(N,D)
    }

    learning_rate = 1e-5
    for t in range(50)
        out = sess.run([loss, grad_w1, grad_w2], feed_dict =
values)

        loss_val, grad_w1_val, grad_w2_val = out
        values[w1] = learning_rate* grad_w1_val
        values[w2] = learning_rate* grad_w2_val
```

TensorFlow: Neural Net

Train the network: Run the graph over and over, use gradient to update weights

Problem: copying weights between CPU / GPU each step

```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N,D))
y = tf.placeholder(tf.float32, shape=(N,D))
w1 = tf.placeholder(tf.float32, shape=(D,H))
w2 = tf.placeholder(tf.float32, shape=(H,D))

h = tf.matmul(x, w1)
y_pred = tf.matmul(h, w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff**2, axis=1))
grad_w1, grad_w2 = tf.gradients(loss, [w1, w2])
with tf.Session() as sess
    values = {
        x: np.random.randn(N,D)
        w1: np.random.randn(D,H)
        w2: np.random.randn(H,D)
        y: np.random.randn(N,D)
    }

    learning_rate = 1e-5
    for t in range(50)
        out = sess.run([loss, grad_w1, grad_w2], feed_dict =
values)

        loss_val, grad_w1_val, grad_w2_val = out
        values[w1] = learning_rate* grad_w1_val
        values[w2] = learning_rate* grad_w2_val
```

TensorFlow: Neural Net

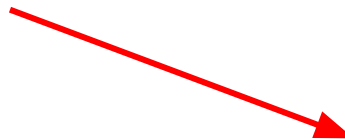
Change w1 and w2 from **placeholder** (fed on each call) to **Variable** (persists in the graph between calls)

```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N,D))
y = tf.placeholder(tf.float32, shape=(N,D))
w1 = tf.Variable(tf.random_normal(D,H))
w2 = tf.Variable(tf.random_normal(H,D))

h = tf.maximun(tf.matmul(x, w1), 0)
y_pred = tf.matmul(h,w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff**2, axis=1))
grad_w1, grad_w2 = tf.gradients(loss, [w1, w2])
learning_rate = 1e-5
new_w1 = tf.assign(w1 - learning_rate* grad_w1)
new_w2 = tf.assign(w2 - learning_rate* grad_w2)
with tf.Session() as sess
    sess.run(tf.global_variables_initializer())
    values = {
        x: np.random.randn(N,D)
        y: np.random.randn(N,D)
    }
    for t in range(50)
        loss_val = sess.run([loss], feed_dict = values)
```

TensorFlow: Neural Net

Add **assign** operations
to update w1 and w2 as
part of the graph!



```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N,D))
y = tf.placeholder(tf.float32, shape=(N,D))
w1 = tf.Variable(tf.random_normal(D,H))
w2 = tf.Variable(tf.random_normal(H,D))

h = tf.maximun(tf.matmul(x, w1), 0)
y_pred = tf.matmul(h,w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff**2, axis=1))
grad_w1, grad_w2 = tf.gradients(loss, [w1, w2])
learning_rate = 1e-5
new_w1 = tf.assign(w1 - learning_rate* grad_w1)
new_w2 = tf.assign(w2 - learning_rate* grad_w2)

with tf.Session() as sess
    sess.run(tf.global_variables_initializer())
    values = {
        x: np.random.randn(N,D)
        y: np.random.randn(N,D)
    }
    for t in range(50)
        loss_val = sess.run([loss], feed_dict = values)
```

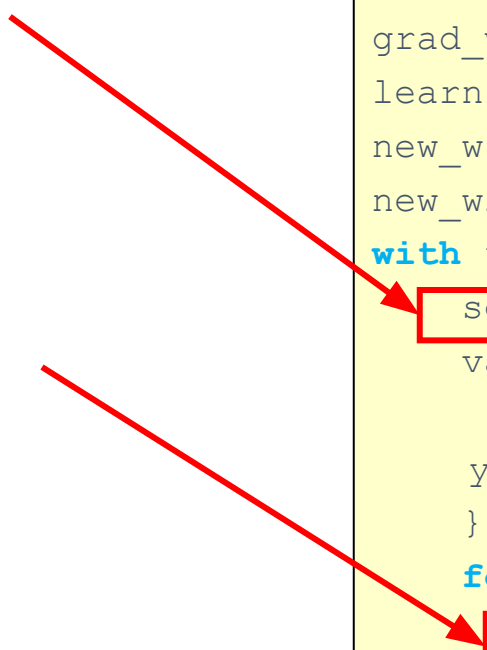
TensorFlow: Neural Net

Run graph once to
initialize w1 and w2

Run many times to train

```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N,D))
y = tf.placeholder(tf.float32, shape=(N,D))
w1 = tf.Variable(tf.random_normal(D,H))
w2 = tf.Variable(tf.random_normal(H,D))

h = tf.matmul(x, w1)
y_pred = tf.matmul(h, w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff**2, axis=1))
grad_w1, grad_w2 = tf.gradients(loss, [w1, w2])
learning_rate = 1e-5
new_w1 = tf.assign(w1 - learning_rate* grad_w1)
new_w2 = tf.assign(w2 - learning_rate* grad_w2)
with tf.Session() as sess
    sess.run(tf.global_variables_initializer())
    values = {
        x: np.random.randn(N,D)
        y: np.random.randn(N,D)
    }
    for t in range(50)
        loss_val = sess.run([loss], feed_dict = values)
```



TensorFlow: Neural Net

Run graph once to
initialize w1 and w2

Run many times to train

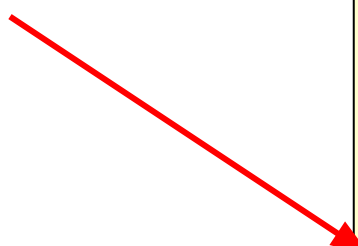
Problem: loss not going
down! Assign calls not
actually being executed!

```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N,D))
y = tf.placeholder(tf.float32, shape=(N,D))
w1 = tf.Variable(tf.random_normal(D,H))
w2 = tf.Variable(tf.random_normal(H,D))

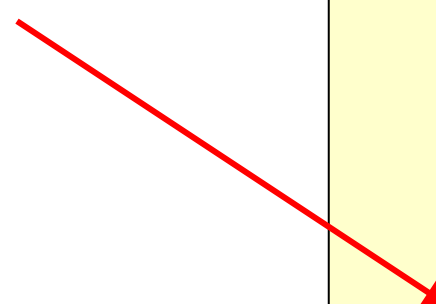
h = tf.maximun(tf.matmul(x, w1), 0)
y_pred = tf.matmul(h,w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff**2, axis=1))
grad_w1, grad_w2 = tf.gradients(loss, [w1, w2])
learning_rate = 1e-5
new_w1 = tf.assign(w1 - learning_rate* grad_w1)
new_w2 = tf.assign(w2 - learning_rate* grad_w2)
with tf.Session() as sess
    sess.run(tf.global_variables_initializer())
    values = {
        x: np.random.randn(N,D)
        y: np.random.randn(N,D)
    }
    for t in range(50)
        loss_val = sess.run([loss], feed_dict = values)
```

TensorFlow: Neural Net

Add dummy graph node
that depends on updates



Tell graph to compute
dummy node



```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N,D))
y = tf.placeholder(tf.float32, shape=(N,D))
w1 = tf.Variable(tf.random_normal(D,H))
w2 = tf.Variable(tf.random_normal(H,D))

h = tf.matmul(x, w1)
y_pred = tf.matmul(h, w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff**2, axis=1))
grad_w1, grad_w2 = tf.gradients(loss, [w1, w2])
learning_rate = 1e-5
new_w1 = tf.assign(w1 - learning_rate* grad_w1)
new_w2 = tf.assign(w2 - learning_rate* grad_w2)
updates = tf.group(new_w1, new_w2)

with tf.Session() as sess:
    sess.run(tf.global_variables_initializer())
    values = {
        x: np.random.randn(N,D),
        y: np.random.randn(N,D)
    }
    for t in range(50):
        loss_val = sess.run([loss, updates], feed_dict = values)
```


TensorFlow: Optimizer

Can use an **optimizer** to compute gradients and update weights

Remember to execute the output of the optimizer!

```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N,D))
y = tf.placeholder(tf.float32, shape=(N,D))
w1 = tf.Variable(tf.random_normal(D,H))
w2 = tf.Variable(tf.random_normal(H,D))

h = tf.matmul(x, w1)
y_pred = tf.matmul(h, w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff**2, axis=1))
learning_rate = 1e-5
optimizer = tf.train.GradientDescentOptimizer(learning_rate)
updates = optimizer.minimize(loss)

with tf.Session() as sess:
    sess.run(tf.global_variables_initializer())
    values = {
        x: np.random.randn(N,D),
        y: np.random.randn(N,D),
    }
    for t in range(50):
        loss_val = sess.run([loss, updates], feed_dict = values)
```

TensorFlow: Loss

Use predefined
common losses



```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N,D))
y = tf.placeholder(tf.float32, shape=(N,D))
w1 = tf.Variable(tf.random_normal(D,H))
w2 = tf.Variable(tf.random_normal(H,D))

h = tf.matmul(x, w1)
y_pred = tf.matmul(h, w2)
diff = y_pred - y
loss = tf.losses.mean_squared_error(y_pred, y)
learning_rate = 1e-5
optimizer = tf.train.GradientDescentOptimizer(learning_rate)
updates = optimizer.minimize(loss)

with tf.Session() as sess:
    sess.run(tf.global_variables_initializer())
    values = {
        x: np.random.randn(N,D),
        y: np.random.randn(N,D),
    }
    for t in range(50):
        loss_val = sess.run([loss, updates], feed_dict = values)
```

TensorFlow: Layers

Use Xavier
initializer

tf.layers automatically
sets up weight and
(and bias) for us!

```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N,D))
y = tf.placeholder(tf.float32, shape=(N,D))
init = tf.contrib.layers.xavier_initializer()
h = tf.layers.dense(input=x, units=H, activation = tf.nn.relu,
kernel_initializer=init)
y_pred = tf.layers.dense(input=h, units=D,
kernel_initializer=init)
loss = tf.losses.mean_squared_error(y_pred, y)
learning_rate = 1e-5
optimizer = tf.train.GradientDescentOptimizer(learning_rate)
updates = optimizer.minimize(loss)
with tf.Session() as sess
    sess.run(tf.global_variables_initializer())
    values = {
        x: np.random.randn(N,D)
        y: np.random.randn(N,D)
    }
    for t in range(50)
        loss_val = sess.run([loss, updates], feed_dict = values)
```



PyTorch

Deep learning and Its Applications in Python



PyTorch: Three Levels of Abstraction

TensorFlow equivalent

Tensor: Imperative
ndarray, but runs on GPU

Numpy array

Variable: Node in a computational
graph; stores data and gradient

Tensor, Variable,
Placeholder

Module: A neural network layer;
may store state or learnable weights

tf.layers, or TFSlim, or
TFLearn, or Sonnet, or

PyTorch: Tensors

PyTorch Tensors are just like numpy arrays, but they can run on GPU.

No built-in notion of computational graph, or gradients, or deep learning.

Here we fit a two-layer net using PyTorch Tensors

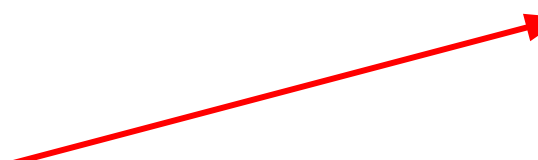
```
import torch
from torch.autograd import Variable
dtype = torch.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in).type(dtype)
y = torch.randn(N, D_out).type(dtype)
w1 = torch.randn(D_in, H).type(dtype)
w2 = torch.randn(H, D_out).type(dtype)

learning_rate = 1e-5
for t in range(50):
    h = x.mm(w1)
    hr_relu = h.clamp(min=0)
    y_pred = hr_relu.mm(w2)
    loss = (y_pred - y).pow(2).sum()
    grad_y_pred = 2.0*(y_pred - y)
    grad_w2 = hr_relu.t().mm(grad_y_pred)
    grad_h_relu = grad_y_pred.mm(w2.t())
    grad_h = grad_h_relu.clone()
    grad_h[h < 0] = 0
    grad_w1 = x.t().mul(grad_h)
    w1 -= learning_rate*grad_w1
    w2 -= learning_rate*grad_w2
```

PyTorch: Tensors

Create random
tensors for data and
weights



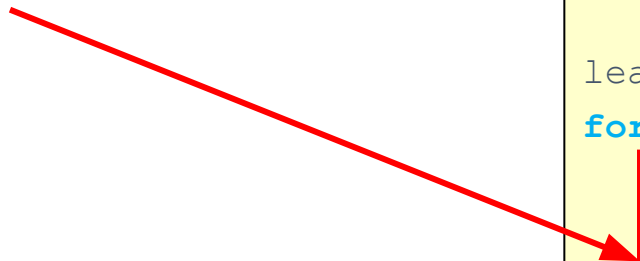
```
import torch
from torch.autograd import Variable
dtype = torch.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in).type(dtype)
y = torch.randn(N, D_out).type(dtype)
w1 = torch.randn(D_in, H).type(dtype)
w2 = torch.randn(H, D_out).type(dtype)

learning_rate = 1e-5
for t in range(50):
    h = x.mm(w1)
    hr_relu = h.clamp(min=0)
    y_pred = hr_relu.mm(w2)
    loss = (y_pred - y).pow(2).sum()
    grad_y_pred = 2.0*(y_pred - y)
    grad_w2 = hr_relu.t().mm(grad_y_pred)
    grad_h_relu = grad_y_pred.mm(w2.t())
    grad_h = grad_h_relu.clone()
    grad_h[h < 0] = 0
    grad_w1 = x.t().mul(grad_h)
    w1 -= learning_reate*grad_w1
    w2 -= learning_reate*grad_w2
```

PyTorch: Tensors

Forward pass:
compute predictions
and loss



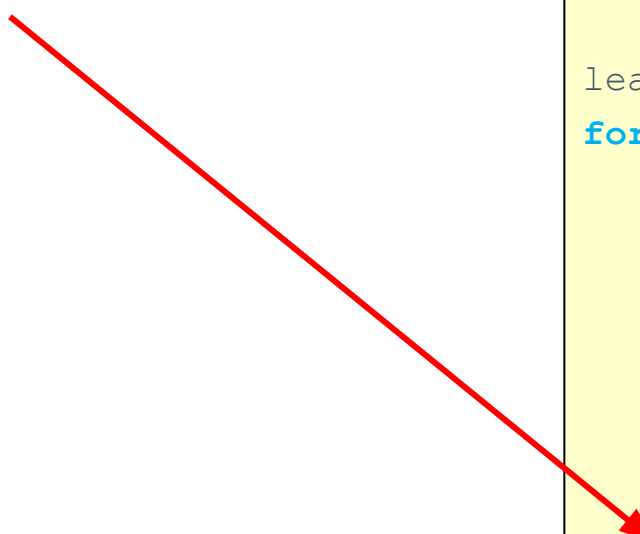
```
import torch
from torch.autograd import Variable
dtype = torch.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in).type(dtype)
w1 = torch.randn(D_in, H).type(dtype)
y = torch.randn(N, D_out).type(dtype)
w2 = torch.randn(H, D_out).type(dtype)

learning_rate = 1e-5
for t in range(50):
    h = x.mm(w1)
    hr_relu = h.clamp(min=0)
    y_pred = hr_relu.mm(w2)
    loss = (y_pred - y).pow(2).sum()
    grad_y_pred = 2.0*(y_pred - y)
    grad_w2 = hr_relu.t().mm(grad_y_pred)
    grad_h_relu = grad_y_pred.mm(w2.t())
    grad_h = grad_h_relu.clone()
    grad_h[h < 0] = 0
    grad_w1 = x.t().mul(grad_h)
    w1 -= learning_rate*grad_w1
    w2 -= learning_rate*grad_w2
```


PyTorch: Tensors

Backward pass:
manually compute
gradients



```
import torch
from torch.autograd import Variable
dtype = torch.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in).type(dtype)
y = torch.randn(N, D_out).type(dtype)
w1 = torch.randn(D_in, H).type(dtype)
w2 = torch.randn(H, D_out).type(dtype)

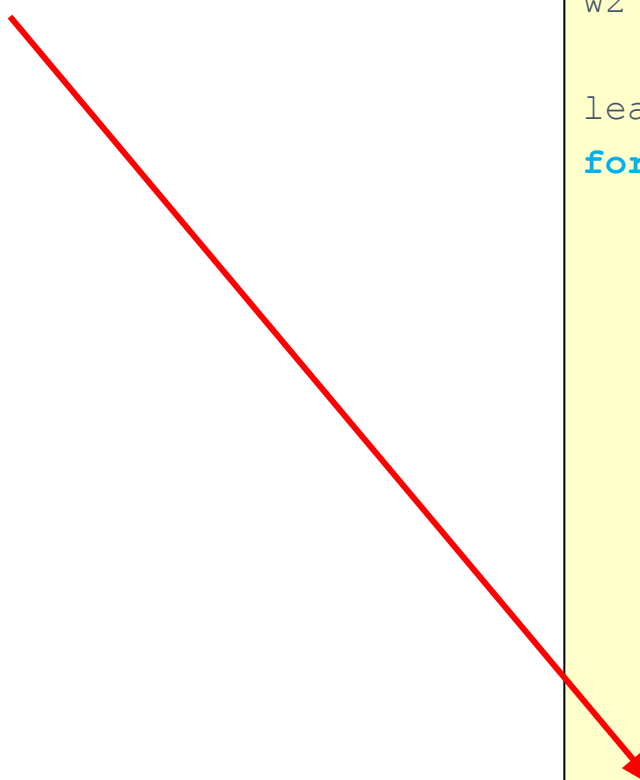
learning_rate = 1e-5
for t in range(50):
    h = x.mm(w1)
    hr_relu = h.clamp(min=0)
    y_pred = hr_relu.mm(w2)
    loss = (y_pred - y).pow(2).sum()

    grad_y_pred = 2.0 * (y_pred - y)
    grad_w2 = hr_relu.t().mm(grad_y_pred)
    grad_h_relu = grad_y_pred.mm(w2.t())
    grad_h = grad_h_relu.clone()
    grad_h[h < 0] = 0
    grad_w1 = x.t().mul(grad_h)

    w1 -= learning_rate * grad_w1
    w2 -= learning_rate * grad_w2
```

PyTorch: Tensors

Gradient descent
step on weights



```
import torch
from torch.autograd import Variable
dtype = torch.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in).type(dtype)
y = torch.randn(N, D_out).type(dtype)
w1 = torch.randn(D_in, H).type(dtype)
w2 = torch.randn(H, D_out).type(dtype)

learning_rate = 1e-5
for t in range(50):
    h = x.mm(w1)
    hr_relu = h.clamp(min=0)
    y_pred = hr_relu.mm(w2)
    loss = (y_pred - y).pow(2).sum()
    grad_y_pred = 2.0*(y_pred - y)
    grad_w2 = hr_relu.t().mm(grad_y_pred)
    grad_h_relu = grad_y_pred.mm(w2.t())
    grad_h = grad_h_relu.clone()
    grad_h[h < 0] = 0
    grad_w1 = x.t().mul(grad_h)
    w1 -= learning_rate*grad_w1
    w2 -= learning_rate*grad_w2
```

PyTorch: Tensors

To run on GPU, just
cast tensors to a
cuda datatype!

```
import torch
from torch.autograd import Variable
dtype = torch.cuda.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in).type(dtype)
y = torch.randn(N, D_out).type(dtype)
w1 = torch.randn(D_in, H).type(dtype)
w2 = torch.randn(H, D_out).type(dtype)

learning_rate = 1e-5
for t in range(50):
    h = x.mm(w1)
    hr_relu = h.clamp(min=0)
    y_pred = hr_relu.mm(w2)
    loss = (y_pred - y).pow(2).sum()
    grad_y_pred = 2.0*(y_pred - y)
    grad_w2 = hr_relu.t().mm(grad_y_pred)
    grad_h_relu = grad_y_pred.mm(w2.t())
    grad_h = grad_h_relu.clone()
    grad_h[h < 0] = 0
    grad_w1 = x.t().mul(grad_h)
    w1 -= learning_rate*grad_w1
    w2 -= learning_rate*grad_w2
```

PyTorch: Autograd

A PyTorch **Variable** is a node in a computational graph

x.data is a Tensor

x.grad is a Variable of gradients
(same shape as x.data)

x.grad.data is a Tensor of
gradients

```
import torch
from torch.autograd import Variable
dtype = torch.cuda.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

learning_rate = 1e-5
for t in range(50):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()
    if w1.grad:
        w1.grad.data.zero_()
    if w2.grad:
        w2.grad.data.zero_()
    loss.backward()
    w1 -= learning_rate * w1.grad.data
    w2 -= learning_rate * w2.grad.data
```

PyTorch: Autograd

PyTorch Tensors and Variables have the same API!

Variables remember how they were created (for backprop)

```
import torch
from torch.autograd import Variable
dtype = torch.cuda.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

learning_rate = 1e-5
for t in range(50):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()
    if w1.grad:
        w1.grad.data.zero_()
    if w2.grad:
        w2.grad.data.zero_()
    loss.backward()
    w1 -= learning_rate * w1.grad.data
    w2 -= learning_rate * w2.grad.data
```

PyTorch: Autograd

We will not want
gradients (of loss)
with respect to data

Do want gradients
with respect to
weights

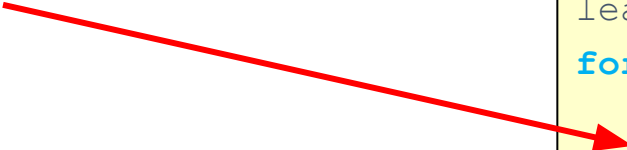
```
import torch
from torch.autograd import Variable
dtype = torch.cuda.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad= False)
y = Variable(torch.randn(N, D_out), requires_grad= False)
w1 = Variable(torch.randn(D_in, H), requires_grad= True)
w2 = Variable(torch.randn(H, D_out), requires_grad= True)

learning_rate = 1e-5
for t in range(50):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()
    if w1.grad:
        w1.grad.data.zero_()
    if w2.grad:
        w2.grad.data.zero_()
    loss.backward()
    w1 -= learning_rate * w1.grad.data
    w2 -= learning_rate * w2.grad.data
```

PyTorch: Autograd

Forward pass looks exactly the same as the Tensor version, but everything is a variable now



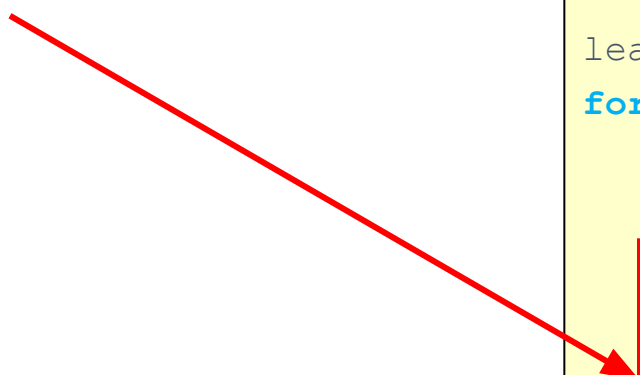
```
import torch
from torch.autograd import Variable
dtype = torch.cuda.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

learning_rate = 1e-5
for t in range(50):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()
    if w1.grad:
        w1.grad.data.zero_()
    if w2.grad:
        w2.grad.data.zero_()
    loss.backward()
    w1 -= learning_rate * w1.grad.data
    w2 -= learning_rate * w2.grad.data
```

PyTorch: Autograd

Compute gradient of
loss with respect to
w1 and w2 (zero out
grads first)



```
import torch
from torch.autograd import Variable
dtype = torch.cuda.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

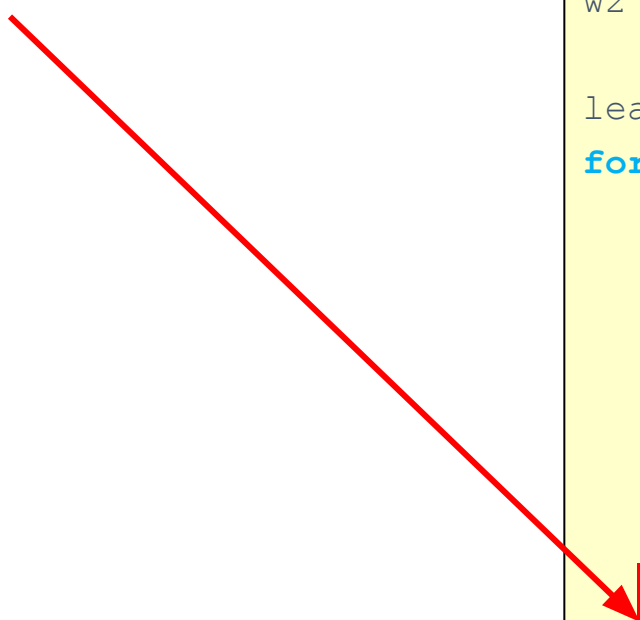
learning_rate = 1e-5
for t in range(50):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    if w1.grad:
        w1.grad.data.zero_()
    if w2.grad:
        w2.grad.data.zero_()
    loss.backward()

    w1 -= learning_rate * w1.grad.data
    w2 -= learning_rate * w2.grad.data
```


PyTorch: Autograd

Make gradient
step on weights



```
import torch
from torch.autograd import Variable
dtype = torch.cuda.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

learning_rate = 1e-5
for t in range(50):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()
    if w1.grad:
        w1.grad.data.zero_()
    if w2.grad:
        w2.grad.data.zero_()
    loss.backward()
    w1 -= learning_rate * w1.grad.data
    w2 -= learning_rate * w2.grad.data
```

PyTorch: nn

Higher-level wrapper for working
with neural nets

Similar to Keras and friends ... but
only one, and it's good =)

```
import torch
from torch.autograd import Variable
dtype = torch.cuda.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
Loss_fn = torch.nn.MSELoss(size_average=False)
learning_rate = 1e-5

for t in range(50):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)
    optimizer.zero_grad()
    loss.backward()
    for param in model.parameters():
        param.data -= learning_rate * param.grad.data
```

PyTorch: nn

Define our model as
a sequence of layers

nn also defines
common loss
functions

```
import torch
from torch.autograd import Variable
dtype = torch.cuda.FloatTensor

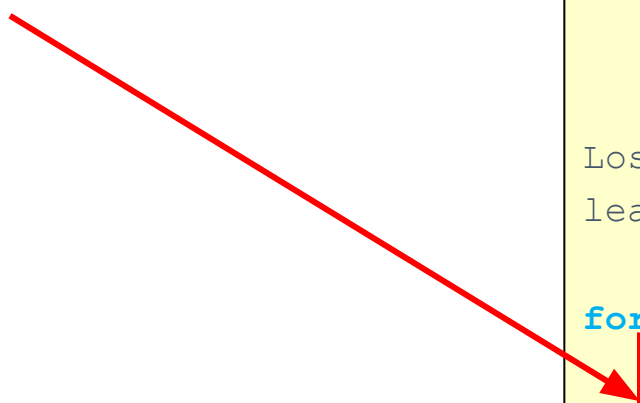
N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
loss_fn = torch.nn.MSELoss(size_average=False)
learning_rate = 1e-5

for t in range(50):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)
    optimizer.zero_grad()
    loss.backward()
    for param in model.parameters():
        param.data -= learning_rate * param.grad.data
```

PyTorch: nn

Forward pass: feed data to model, and prediction to loss function



```
import torch
from torch.autograd import Variable
dtype = torch.cuda.FloatTensor

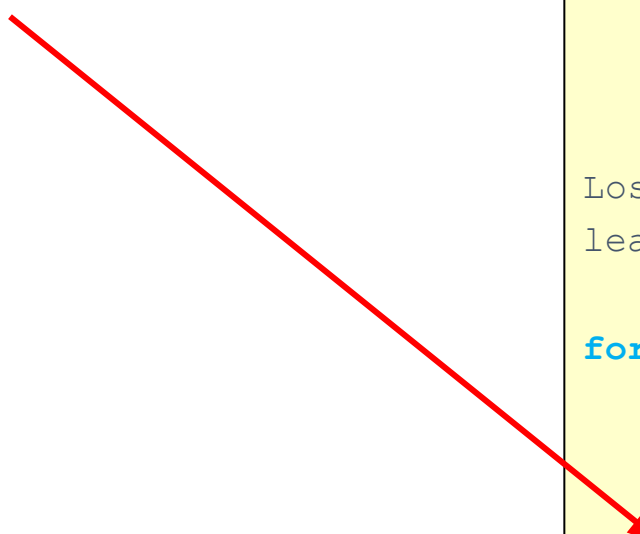
N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
Loss_fn = torch.nn.MSELoss(size_average=False)
learning_rate = 1e-5

for t in range(50):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)
    optimizer.zero_grad()
    loss.backward()
    for param in model.parameters():
        param.data -= learning_rate * param.grad.data
```

PyTorch: nn

Backward pass:
compute all
gradients



```
import torch
from torch.autograd import Variable
dtype = torch.cuda.FloatTensor

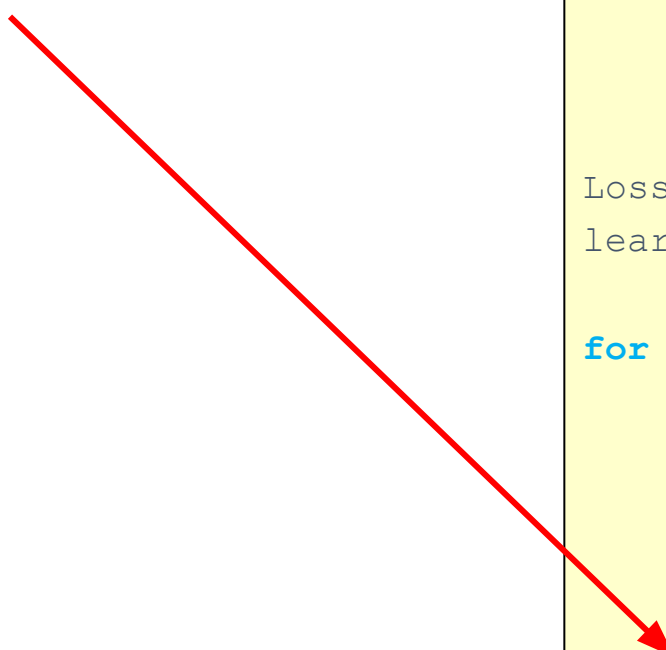
N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
Loss_fn = torch.nn.MSELoss(size_average=False)
learning_rate = 1e-5

for t in range(50):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)
    optimizer.zero_grad()
    loss.backward()
    for param in model.parameters():
        param.data -= learning_rate * param.grad.data
```

PyTorch: nn

Make gradient step
on each model
parameter



```
import torch
from torch.autograd import Variable
dtype = torch.cuda.FloatTensor

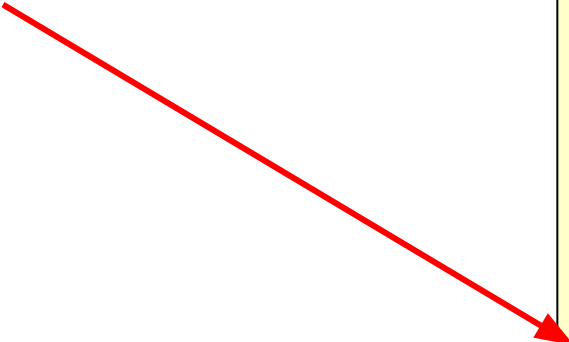
N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
Loss_fn = torch.nn.MSELoss(size_average=False)
learning_rate = 1e-5

for t in range(50):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)
    optimizer.zero_grad()
    loss.backward()
    for param in model.parameters():
        param.data -= learning_rate * param.grad.data
```

PyTorch: optm

Make gradient
step on weights



```
import torch
from torch.autograd import Variable
dtype = torch.cuda.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)

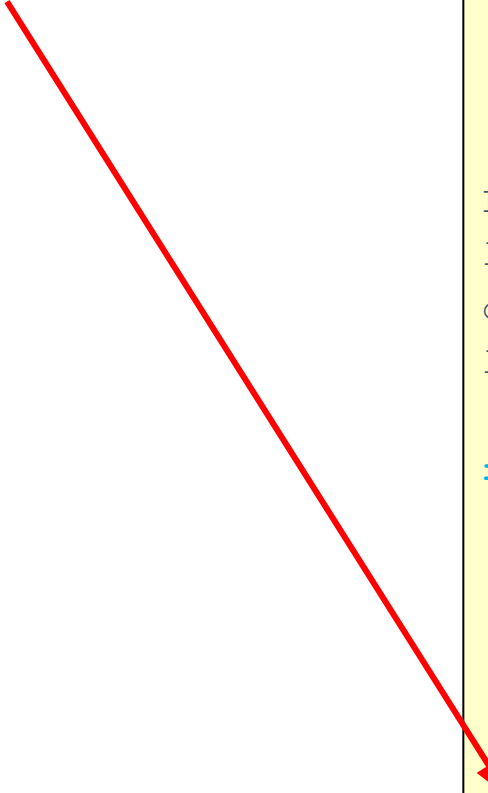
model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
Loss_fn = torch.nn.MSELoss(size_average=False)
learning_rate = 1e-5
optimizer = torch.optim.Adam(model.parameters(),
                               lr=learning_rate)

for t in range(50):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)
    optimizer.zero_grad()
    loss.backward()

    optimizer.step()
```

PyTorch: optm

Update all parameters
after computing gradients



```
import torch
from torch.autograd import Variable
dtype = torch.cuda.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
loss_fn = torch.nn.MSELoss(size_average=False)
learning_rate = 1e-5
optimizer = torch.optim.Adam(model.parameters(),
                               lr=learning_rate)

for t in range(50):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)
    optimizer.zero_grad()
    loss.backward()

    optimizer.step()
```


Static vs Dynamic Graphs

TensorFlow: Build graph once, then run many times (**static**)

```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N,D))
y = tf.placeholder(tf.float32, shape=(N,D))
w1 = tf.Variable(tf.random_normal(D,H))
w2 = tf.Variable(tf.random_normal(H,D))
h = tf.matmul(x, w1)
y_pred = tf.matmul(h, w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff**2, axis=1))
grad_w1, grad_w2 = tf.gradients(loss, [w1, w2])
learning_rate = 1e-5
new_w1 = tf.assign(w1 - learning_rate* grad_w1)
new_w2 = tf.assign(w2 - learning_rate* grad_w2)
with tf.Session() as sess:
    sess.run(tf.global_variables_initializer())
    values = {
        x: np.random.randn(N,D),
        y: np.random.randn(N,D)
    }
    for t in range(50):
        loss_val = sess.run([loss], feed_dict = values)
```

Build
graph

Run
each
iteration

PyTorch: Each forward pass defines a new graph (**dynamic**)

```
import torch
from torch.autograd import Variable
dtype = torch.cuda.FloatTensor
N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

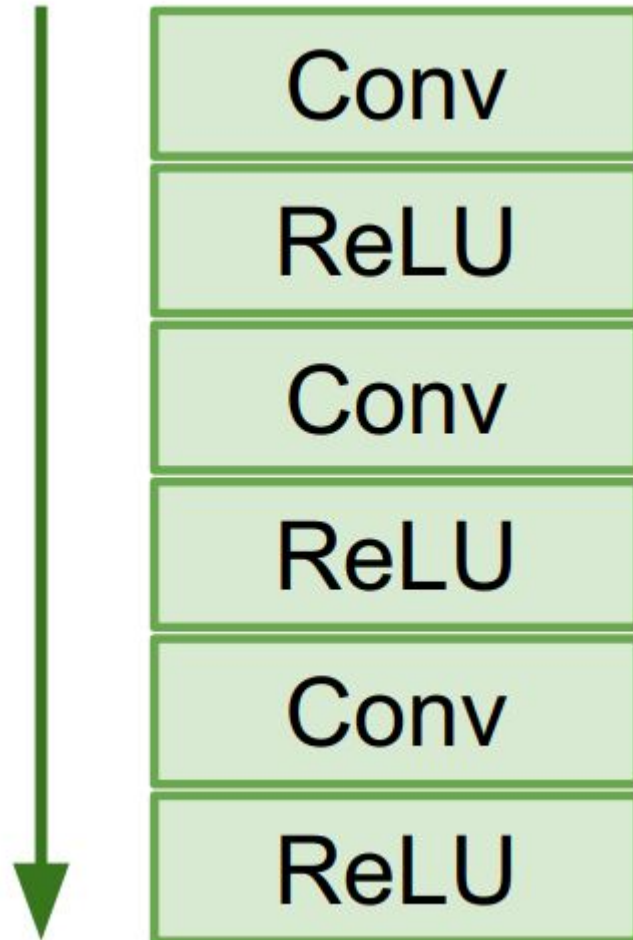
learning_rate = 1e-5
for t in range(50):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()
    if w1.grad:
        w1.grad.data.zero_()
    if w2.grad:
        w2.grad.data.zero_()
    loss.backward()
    w1 -= learning_rate * w1.grad.data
    w2 -= learning_rate * w2.grad.data
```

New Graph
each
iteration

Static vs Dynamic Graphs: Optimization

The graph you wrote

With static graphs, framework can **optimize** the graph for you before it runs!



Equivalent graph with **fused operations**



Static vs Dynamic: Serialization

Static

Once graph is built,
can **serialize** it and
run it without the code
that built the graph!

Dynamic

Graph building and execution
are intertwined, so always
need to keep code around

Static vs Dynamic: Conditional

$$y = \begin{cases} w1 * x & \text{if } z > 0 \\ w2 * x & \text{otherwise} \end{cases}$$

```
N, D, H = 3, 4, 5
x = Variable(torch.randn(N,D))
w1 = Variable(torch.randn(D,H))
w2 = Variable(torch.randn(D,H))

Z = 10
if x > 10:
    y = x.mm(w1)
else:
    y = x.mm(w2)
```

PyTorch: Normal Python

TensorFlow: Special TF control flow operator!

```
N, D, H = 3, 4, 5
x = tf.placeholder(tf.float32, shape=(N,D))
z = tf.placeholder(tf.float32, shape=None)
w1 = tf.placeholder(tf.float32, shape=(D,H))
w2 = tf.placeholder(tf.float32, shape=(D,H))

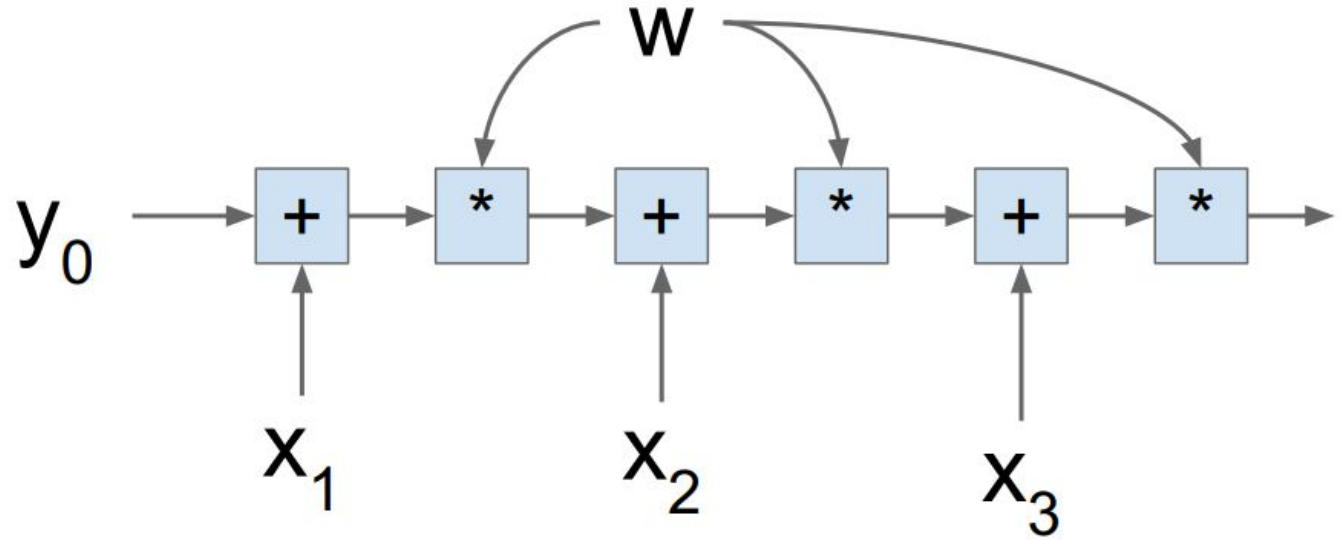
def f1: return tf.matmul(x, w1)
def f2: return tf.matmul(x, w2)

y = tf.cond(tf.less(z,0), f1, f2)

with tf.Session() as sess
    values = {
        x: np.random.randn(N,D)
        w1: np.random.randn(D,H)
        w2: np.random.randn(H,D)
        z: 10}
```

Static vs Dynamic: Loops

$$y_t = (y_{t-1} + x_t) * w$$

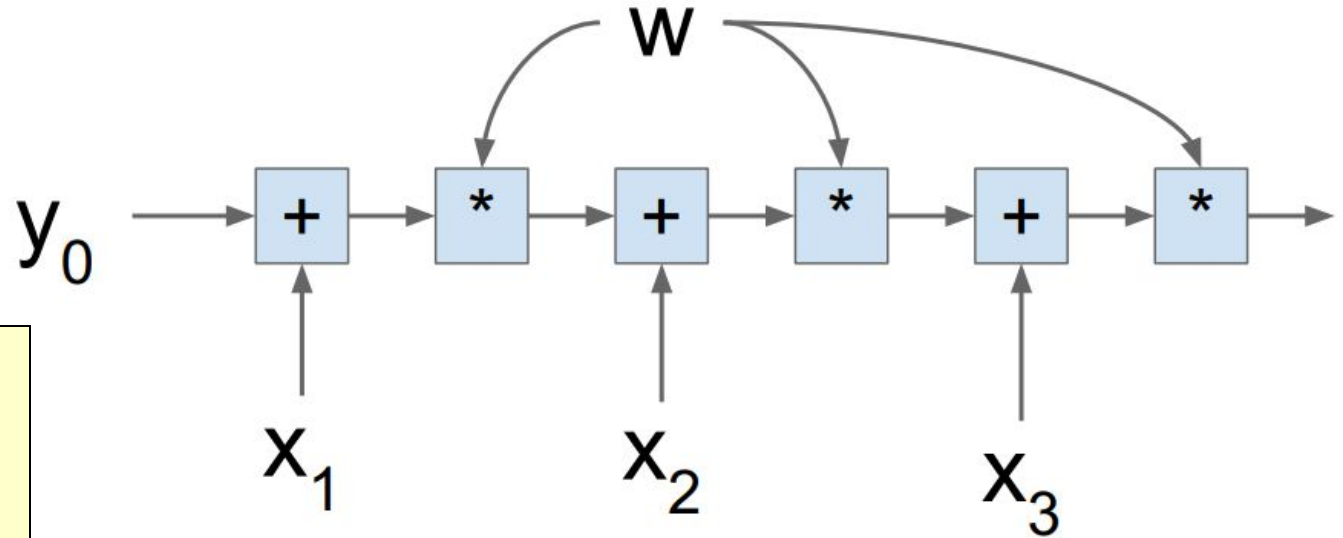


Static vs Dynamic: Loops

$$y_t = (y_{t-1} + x_t) * w$$

```
T, D = 3, 4
y0 = Variable(torch.randn(D))
x = Variable(torch.randn(T,D))
w = Variable(torch.randn(D))

y = [y0]
for t in range(T) > 10:
    prev_y = y[-1]
    next_y = (prev_y + x[t])*w
    y.append(next_y)
```



Static vs Dynamic: Loops

$$y_t = (y_{t-1} + x_t) * w$$

```
T, D = 3, 4
y0 = Variable(torch.randn(D))
x = Variable(torch.randn(T,D))
w = Variable(torch.randn(D))

y = [y0]
for t in range(T) > 10:
    prev_y = y[-1]
    next_y = (prev_y + x[t])*w
    y.append(next_y)
```

```
T, N, D = 3, 4, 5
x = tf.placeholder(tf.float32, shape=(T,D))
y0 = tf.placeholder(tf.float32, shape=(D,))
w = tf.placeholder(tf.float32, shape=(D,))
def f (prev_y, cur_x):
    return (prev_y + cur_x)*w

y = tf.foldl(f, x, y)

with tf.Session() as sess
    values = {
        x: np.random.randn(T,D)
        y0: np.random.randn(D)
        w: np.random.randn(D)
    }
    out = sess.run(y, feed_dict = values)
```

Dynamic Graphs in Tensorflow

TensorFlow Fold make dynamic graphs easier in TensorFlow through **dynamic batching**



Outline

- What's Keras?
 - What's special about it?
 - TensorFlow integration
- How to use Keras
 - 3 API styles
- Distributed, multi-GPU, and TPU training
- Eager execution (a.k.a define-by-run, a.k.a. dynamic graphs)

What's Keras?



Keras: an API for specifying &
training differentiable programs

What's Keras?



Keras API

TensorFlow, Theano, CNTK,
MXNet...

CPU

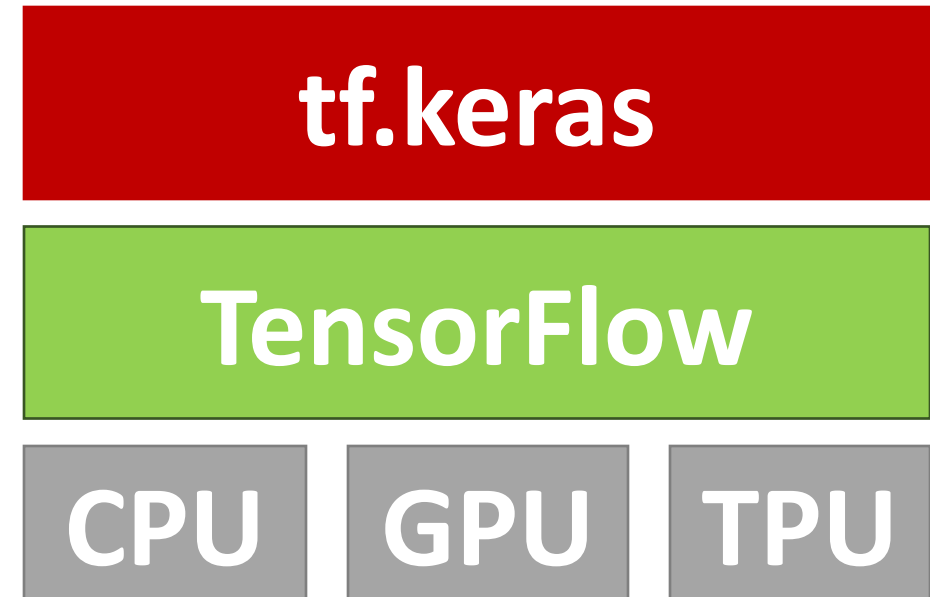
GPU

TPU

Keras is the official high-level API of TensorFlow



- tensorflow.keras (tf.keras) module
- Part of core TensorFlow since v1.4
- Full Keras API
- Better optimized for TF
- Better integration with TF-specific features
 - Estimator API
 - Eager execution
 - etc.





What is special about Keras?

- A focus on user experience.
- Large adoption in the industry and research community.
- Multi-backend, multi-platform.
- Easy productization of models.

The Keras user experience

Keras is an API designed for human beings, not machines. Keras follows best practices for reducing cognitive load: it offers consistent & simple APIs, it minimizes the number of user actions required for common use cases, and it provides clear and actionable feedback upon user error.

This makes Keras easy to learn and easy to use. As a Keras user, you are more productive, allowing you to try more ideas than your competition, faster -- which in turn helps you win machine learning competitions.

This ease of use does not come at the cost of reduced flexibility: because Keras integrates with lower-level deep learning languages (in particular TensorFlow), it enables you to implement anything you could have built in the base



How to use Keras: Introduction

Deep learning and Its Applications in Python



Three API Styles

- The Sequential Model
 - Dead simple
 - Only for single-input, single-output, sequential layer stacks
 - Good for 70+% of use cases
- The functional API
 - Like playing with Lego bricks
 - Multi-input, multi-output, arbitrary static graph topologies
 - Good for 95% of use cases
- Model subclassing
- Maximum flexibility
- Larger potential error surface



Keras is multi-backend, multi-platform

- Develop in Python, R
- Run the same code with...
 - TensorFlow
 - CNTK
 - Theano
 - MXNet
 - PlaidML
- CPU, NVIDIA GPU, AMD GPU, TPU...



The Sequential API

```
import keras
from keras import layers

model = keras.Sequential()
model.add(layers.Dense(20, activation='relu', input_shape=(10,)))
model.add(layers.Dense(20, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))

model.fit(x, y, epochs=10, batch_size=32)
```



The functional API

```
import keras
from keras import layers

inputs = keras.Input(shape=(10,))
x = layers.Dense(20, activation='relu')(x)
x = layers.Dense(20, activation='relu')(x)
outputs = layers.Dense(10, activation='softmax')(x)

model = keras.Model(inputs, outputs)
model.fit(x, y, epochs=10, batch_size=32)
```

Model subclassing



```
import keras
from keras import layers

class MyModel(keras.Model):

    def __init__(self):
        super(MyModel, self).__init__()
        self.dense1 = layers.Dense(20, activation='relu')
        self.dense2 = layers.Dense(20, activation='relu')
        self.dense3 = layers.Dense(10, activation='softmax')

    def call(self, inputs):
        x = self.dense1(x)
        x = self.dense2(x)
        return self.dense3(x)

model = MyModel()
model.fit(x, y, epochs=10, batch_size=32)
```



Distributed, Multi-GPU and TPU Training

Deep learning and Its Applications in Python



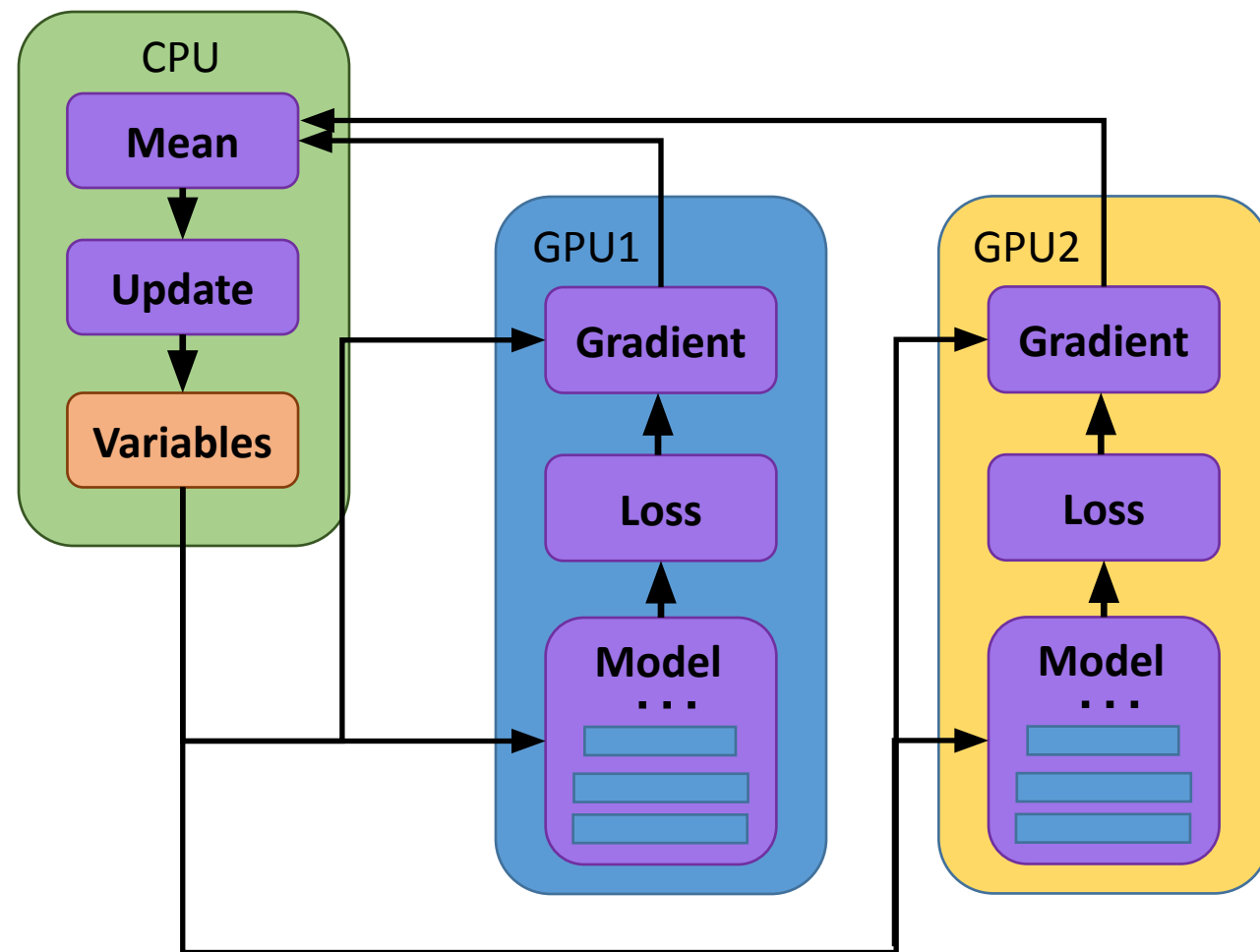
Built-in multi-GPU support

```
import tensorflow as tf
from keras.applications import Xception
from keras.utils import multi_gpu_model
# Instantiate the base model
# Replicates the model on 8 GPUs
# This assumes that your machine has 8 available GPUs
parallel_model = multi_gpu_model(model, gpus=8)
parallel_model.compile( loss='categorical_crossentropy',
                      optimizer='rmsprop')
# This 'fit' call will be distributed on 8 gpus
# Since the batch size is 256, so each gpu will process 32 samples
parallel_model.fit(x, y, epochs=100, batch_size=256)
```



Built-in multi-GPU support

```
import tensorflow as tf
from keras.applications import Xception
from keras.utils import multi_gpu_model
# Instantiate the base model
# Replicates the model on 8 GPUs
# This assumes that your machine has 8
# available GPUs
parallel_model = multi_gpu_model(model,
                                gpus=8)
parallel_model.compile(
    loss='categorical_crossentropy',
    optimizer='rmsprop')
# This 'fit' call will be distributed on 8
# gpus
# Since the batch size is 256, so each gpu
# will process 32 samples
parallel_model.fit(x, y, epochs=100,
                  batch_size=256)
```





Eager Execution

Deep learning and Its Applications in Python

Understanding deferred (symbolic) vs. eager (imperative)

- **Deferred:** you use Python to build a computation graph that gets executed later
- **Eager:** the Python runtime is the execution runtime (like Numpy)

In short:

- Symbolic tensors **don't have a value** in your Python code (yet)
- Eager tensors **have a value** in your Python code
- With eager execution, you can use value-dependent dynamic topologies

The Keras functional API and Sequential API work with eager execution

```
import keras
from keras import layers
import tensorflow.contrib.eager as tfe

tfe.enable_eager_execution()

inputs = keras.Input(shape=(10,))
x = layers.Dense(20, activation='relu')(x)
x = layers.Dense(20, activation='relu')(x)
outputs = layers.Dense(10, activation='softmax')(x)

model = keras.Model(inputs, outputs)

preds = model(x) # Works on *value arrays*!
model.fit(x, y, epochs=10, batch_size=32) # The Model API works too.
```

The Keras functional API and Sequential API work with eager execution

```
class DynamicLayer(tf.keras.Layer):  
  
    ...  
  
    def call(self, x):  
        print(x) # debug!  
        if tf.reduce_sum(x) == 0:  
            # dynamic if!  
            return x  
  
        for t in x.shape[1]:  
            # dynamically iterate over time axis!  
            ...  
  
        return outputs
```

Remember: use the
right tool (API) for
the job

- <https://filebox.ece.vt.edu/~jbhuang/teaching/ece6554/sp17/lectures/>
- <https://elitedatascience.com/keras-tutorial-deep-learning-in-python>
- <https://lilianweng.github.io/lil-log/2017/06/21/an-overview-of-deep-learning.html#recurrent-neural-network>
- <https://ujjwalkarn.me/2016/08/11/intuitive-explanation-convnets/>